Annales NFP 107, CNAM Paris

Ingénierie et Optimisation de Bases de données

1 NFP 107 FOD, 12 Février 2012 **4**
- 1.1 Algèbre relationnelle et SQL (10 points) . . . 4
- 1.2 Indexation (3 points) . . . 7
- 1.3 Optimisation (3 points) . . . 9
- 1.4 Concurrence (4 points) . . . 10

2 NFP 107 FOD, 12 avril 2012 **12**
- 2.1 Algèbre relationnelle et SQL (10 points) . . . 12
- 2.2 Indexation (3 points) . . . 14
- 2.3 Optimisation (3 points) . . . 16
- 2.4 Concurrence (4 points) . . . 17

3 NFP 107 HTT, 4 Juillet 2012 **19**
- 3.1 Algèbre relationnelle et SQL (10 points) . . . 19
- 3.2 Index (2 points) . . . 21
- 3.3 Optimisation (3 points) . . . 21
- 3.4 Concurrence (5 points) . . . 22

4 NFP 107 HTT, 12 septembre 2012 **24**
- 4.1 Algèbre relationnelle et SQL (9 points) . . . 24
- 4.2 Index (3 points) . . . 24
- 4.3 Optimisation (4 points) . . . 25
- 4.4 Concurrence (4 points) . . . 25

5 NFP 107 FOD, 9 février 2013 **27**
- 5.1 Algèbre relationnelle et SQL (10 points) . . . 27
- 5.2 Index (3 points) . . . 29
- 5.3 Optimisation (4 points) . . . 30
- 5.4 Concurrence (3 points) . . . 30

6 NFP 107 FOD, 16 avril 2013 **31**
- 6.1 Algèbre relationnelle et SQL (10 points) . . . 31
- 6.2 Index (2 points) . . . 32
- 6.3 Optimisation (4 points) . . . 33
- 6.4 Concurrence (4 points) . . . 33

7 NFP 107 HTT, Juin 2013 **35**
7.1 Algèbre relationnelle et SQL (10 points) 35
7.2 Indexation (3 points) 37
7.3 Optimisation (4 points) 40
7.4 Concurrence (3 points) 41

8 NFP 107 HTT, Septembre 2013 **43**
8.1 Algèbre relationnelle et SQL (10 points) 43
8.2 Indexation (3 points) 44
8.3 Optimisation (4 points) 45
8.4 Concurrence (3 points) 46

9 NFP 107 FOD, 12 février 2014 **47**
9.1 Algèbre relationnelle et SQL (10 points) 47
9.2 Indexation (3 points) 49
9.3 Optimisation (4 points) 49
9.4 Concurrence (3 points) 50

10 NFP 107 FOD, 9 avril 2014 **51**
10.1 Algèbre relationnelle et SQL (10 points) 51
10.2 Indexation (3 points) 53
10.3 Optimisation (4 points) 53
10.4 Concurrence (3 points) 54

11 NFP 107 HTO, 18 juin 2014 **55**
11.1 Algèbre relationnelle et SQL (10 points) 55
11.2 Indexation (3 points) 57
11.3 Optimisation (4 points) 58
11.4 Concurrence (4 points) 59

12 NFP 107 HTT, 10 Septembre 2014 **61**
12.1 Algèbre relationnelle et SQL (10 points) 61
12.2 Index et Optimisation (5 points) 61
12.3 Concurrence (5 points) 62

13 NFP 107 HTT, 24 juin 2015 **64**
13.1 Algèbre relationnelle et SQL (10 points) 64
13.2 Index (3 points) 66
13.3 Optimisation (4 points) 67
13.4 Concurrence (3 points) 68

14 NFP 107 HTT, Septembre 2015 **69**
14.1 Algèbre relationnelle et SQL (10 points) 69
14.2 Index (3 points) 71
14.3 Optimisation (4 points) 72
14.4 Concurrence (3 points) 73

1 | NFP 107 FOD, 12 Février 2012

1.1 Algèbre relationnelle et SQL (10 points)

Voici une partie de la base de données utilisée pour gérer des œuvres dans un musée:

Oeuvre(id_oeuvre, nom, id_auteur, valeur_assurance, date_acquisition, lieu_exposition, description)
Auteur(id_auteur, nom, prenom, date_naissance, date_deces)
Expert(id_expert, nom, prenom, domaine_expertise)
Expertise(id_oeuvre, id_expert, date_evaluation, valeur)

Informations supplémentaires : Des experts expertisent des œuvres. Plusieurs experts peuvent expertiser une même œuvre. À partir de ces expertises, une valeur est déduite pour l'assurance de l'œuvre.

1.1.1 Pour chaque table, identifier les clés primaires et les clés étrangères. (1 point)
Correction :

- Clés primaires
  Table `Oeuvre : id_oeuvre`
  Table `Auteur : id_auteur`
  Table `Expert : id_expert`
  Table `Expertise : (id_oeuvre, id_expert)`
  ou éventuellement (selon interprétation) `Expertise : (id_oeuvre, id_expert, date_evaluation)`
- Clés étrangères :
  `Oeuvre[id_auteur]` $\subseteq$ `Auteur[id_auteur]`
  `Expertise[id_oeuvre]` $\subseteq$ `Oeuvre[id_oeuvre]`
  `Expertise[id_expert]` $\subseteq$ `Expert[id_expert]`

1.1.2 Donner des commandes SQL permettant de créer les tables **Oeuvre** et **Auteur**. (1 point)

Correction :

CREATE TABLE Auteur (
id_auteur **INTEGER NOT NULL**,
nom **VARCHAR2**(20),

prenom **VARCHAR2**(20),
date_naissance **DATE**,
date_deces **DATE**,
CONSTRAINT Auteur_PK **PRIMARY KEY** (id_auteur)
);

CREATE TABLE Oeuvre (
id_oeuvre **INTEGER NOT NULL**,
nom **VARCHAR2**(40),
id_auteur **INTEGER**,
valeur_assurance **INTEGER**,
date_acquisition **DATE**,
lieu_exposition **VARCHAR2**(30),
description **VARCHAR2**(60),
CONSTRAINT Oeuvre_PK **PRIMARY KEY** (id_oeuvre)
CONSTRAINT Oeuvre_id_auteurKF **FOREIGN KEY** (id_auteur) REFERENCES Auteur (id_auteur)

1.1.3 Donner les identifiants des œuvres dont la valeur assurance est comprise entre 10 000 et 50 000 euros. Algèbre et SQL (1 point)
Correction :

Algèbre:
$\pi_{id_oeuvre}(\sigma_{valeur_assurance \geq 10000 \wedge valeur_assurance \leq 50000}(Oeuvre))$
SQL:

SELECT id_oeuvre **FROM** Oeuvre
WHERE valeur_assurance >= 10000 **AND** valeur_assurance <= 50000;

1.1.4 Donner les noms et prénoms des experts ayant expertisé l'œuvre dont l'identifiant est 345801. Algèbre et SQL. (2 points)

Correction :

Algèbre: Jointure sur l'id de l'expert
$\pi_{nom,prenom}(Expert \bowtie \sigma_{id_oeuvre=345801}(Expertise))$
SQL:

SELECT e.nom, e.prenom **FROM** Expert e, Expertise v
WHERE e.id_expert = v.id_expert **AND** id_oeuvre=345801;

1.1.5 Donner, pour chaque auteur (nom et prénom), la valeur d'assurance (cumulée) de ses œuvres. SQL seulement (1 point)
Correction :

SELECT a.nom, a.prenom, **sum**(valeur_assurance)
FROM Auteur a, Oeuvre o
WHERE o.id_auteur=a.id_auteur
GROUP BY a.id_auteur, a.nom, a.prenom;

1.1.6 Donner les identifiants des œuvres de la *salle 1* (*salle 1* est un lieu d'exposition) ayant été expertisées par au moins deux experts. Algèbre et SQL. (3 points)

Correction :

Algèbre (une solution parmi d'autres):
R_1 (identifiants des œuvres de la salle 1)
$\pi_{id_oeuvre}(\sigma_{lieu_exposition='salle1'}(Oeuvre))$

R_2 (identifiants des œuvres expertisées par au moins deux experts):
$\pi_{id_oeuvre}\left(\sigma_{\substack{id_oeuvre=id_oeuvre2 \\ \wedge id_expert\neq id_expert2}}\left(Expertise \times \rho_{\substack{id_oeuvre\rightarrow id_oeuvre2, \\ id_expert\rightarrow id_expert2}}(Expertise)\right)\right)$
ou une autre solution pour R_2 :
$\pi_{id_oeuvre}\left(\sigma_{id_expert\neq id_expert2}\left(Expertise \bowtie \rho_{\substack{id_expert\rightarrow id_expert2, valeur\rightarrow valeur2, \\ date_evaluation\rightarrow date_evaluation2}}(Expertise)\right)\right)$

R_3 (solution):
$R_1 \cap R_2$

SQL (une solution parmi d'autres):

SELECT o.id_oeuvre
FROM Expertise e1, Expertise e2, Oeuvre o
WHERE e1.id_oeuvre=e2.id_oeuvre **AND** e1.id_expert <> e2.id_expert **AND** e1.id_oeuvre = o.id_oeuvre **AND** o.lieu_exposition='salle 1';

ou

SELECT o.id_oeuvre
FROM Expertise e, Oeuvre o
WHERE e.id_oeuvre = o.id_oeuvre **AND** lieu_exposition='salle 1'
GROUP BY id_oeuvre
HAVING count(e.id_expert) >= 2;

Nous avons considéré qu'un même expert pouvait expertiser plusieurs fois une même oeuvre, la clé primaire de la table `Expertise` étant donc `(id_oeuvre, id_expert, date_evaluation)`.

1.1.7 Donner les identifiants des experts ayant expertisé toutes les œuvres de Edward HOPPER. Algèbre seulement. (1 point)
Correction :

R_1 (les identifiants des œuvres de Edward Hopper) :
$\pi_{id_oeuvre}(Oeuvre \bowtie (\rho_{nom \rightarrow nom_auteur}(\sigma_{nom='Hopper' \wedge prenom='Edward'}(Auteur))))$. Attention ici au renommage de l'attribut nom si on utilise une jointure naturelle.

R_2 (la correction):
$(\pi_{id_expert,id_oeuvre}(Expertise)) \div R_1$

1.2 Indexation (3 points)

On s'intéresse aux designers industriels et on insère successivement les enregistrements suivants dans la table Auteur, selon cet ordre :

id_auteur	**nom**	prenom	date_naissance	date_deces
1	Saarinen	Eero	1910	1961
2	Jacobsen	Arne	1902	1971
3	Le Corbusier	NULL	1887	1965
4	Eames	Charles	1907	1978
5	Bertoia	Harry	1915	1978
6	Panton	Verner	1926	1998
7	Jouin	Patrick	1967	NULL
8	Starck	Philippe	1949	NULL
id_auteur	**nom**	prenom	date_naissance	date_deces
9	Noguchi	Isamu	1904	1988
10	Nelson	George	1908	1986
11	Perriand	Charlotte	1903	1999
12	Prouvé	Jean	1901	1984
13	Aalto	Alvar	1898	1976
14	Gray	Eileen	1878	1976
15	Breuer	Marcel	1902	1981
16	Tallon	Roger	1929	2011

On met en place un index sur l'attribut **nom** de cette table.

1.2.1 Construire l'arbre B+ d'ordre 2 correspondant à cet ensemble d'articles, en respectant l'ordre d'arrivée. Donner les étapes de construction intermédiaires importantes (celles qui produisent un changement de la structure de l'arbre) et distinguer sur les schémas les nœuds de natures différentes (nœuds contenant des clés vs. ceux contenant les enregistrements complets ou autre chose). (1,5 point)

Correction :

La figure 5.1 donne l'arbre B+ d'ordre 2 obtenu. Ici les nœuds internes (en italique) contiennent les clés des enregistrements, alors que les nœuds feuilles (en gras) contiennent les enregistrements complets, ou bien leurs clés plus les pointeurs vers ces enregistrements sur disque.

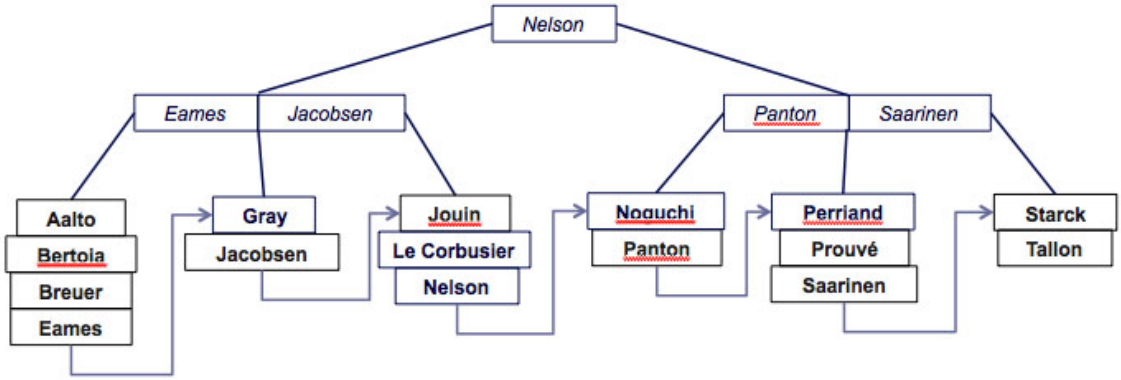


Figure 1.1: Arbre B+ d'ordre 2.

1.2.2 On recherche le prénom des designers dont l'initiale du nom se trouve entre 'H' et 'N'. Combien de pages disque doivent être chargées pour répondre à cette requête en utilisant l'arbre B+ ? Justifier. (1 point)

Correction :

Il s'agit ici d'une requête par intervalle, facilement réalisable avec un arbre B+. On commence par faire une recherche du premier enregistrement dont l'initiale du nom est égale ou immédiatement supérieure à 'H'. On charge le nœud racine 'Nelson' puis son fils gauche puisque 'H' précède 'N'. 'H' se situant entre 'Eames' et 'Jacobsen', on lit ensuite le nœud feuille relatif aux enregistrements 'Gray' et 'Jacobsen'. Jusque là trois nœuds, et donc trois pages, ont été chargés. Dans cette dernière page chargée, 'Jacobsen' est sélectionné car plus grand que 'H' tout en étant plus petit que 'N'. On a trouvé le premier enregistrement répondant à la requête, il suffit d'exploiter le chaînage des feuilles pour trouver les autres enregistrements : on lit ainsi les deux nœuds feuilles suivants, jusqu'à l'enregistrement 'Panton' qui ne rentre par dans l'intervalle recherché. On a trouvé cinq enregistrements répondant à la requête ('Jacobsen', 'Jouin', 'Le Corbusier', 'Nelson' et 'Noguchi'), et lu deux pages disque de nœuds internes, plus trois pages disque de nœuds feuilles. Si les feuilles contiennent les enregistrements complets, alors les prénoms recherchés y figurent et on a fini : **on**

a lu cinq pages disque en tout. Sinon, il faut accéder au fichier des données à partir des pointeurs contenus dans les feuilles. Dans ce cas on doit lire au minimum une page disque (cas optimiste où les cinq enregistrements sélectionnés sont tous sur la même page) et au maximum cinq pages (cas pessismiste où ils sont tous répartis sur des pages différentes) : **on aura donc lu entre six et dix pages en tout**.

1.2.3 Même question en ajoutant aussi une contrainte sur la date de naissance qui doit être comprise entre 1900 et 1950. (0,5 point)

Correction :

La réponse est la même qu'à la question précédente : il n'existe pas d'index particulier sur l'attribut 'date_naissance', donc le parcours de l'arbre et des pages disque sera identique.

1.3 Optimisation (3 points)

1.3.1 Donner une requête SQL correspondant au plain EXPLAIN suivant, sachant que seuls les noms de l'oeuvre et de l'auteur sont projetés (1,5 point) :

```
0  SELECT STATEMENT
1    NESTED LOOP JOIN
2*     TABLE ACCESS FULL        Oeuvre           o
3      TABLE ACCESS BY ROWID    Auteur           a
4*       INDEX UNIQUE SCAN      BTREE_Auteur

(2) valeur_assurance > 50000 AND lieu_exposition = 'Salle 1'
(4) a.id_auteur = o.id_auteur
```

Correction :

SELECT a.nom, o.nom
FROM Auteur a, Oeuvre o
WHERE valeur_assurance > 50000 AND
lieu_exposition = 'Salle 1' AND
a.id_auteur = o.id_auteur

1.3.2 Sachant que nous ajoutons un index de type B+Tree sur **Oeuvre.lieu_exposition**. Modifier le plan EXPLAIN précedent pour intégrer cet index (1,5 point) :

Correction :

```
0  SELECT STATEMENT
1    NESTED LOOP JOIN
2*     TABLE ACCESS BY ROWID    Oeuvre           o
3*       INDEX RANGE SCAN       BTREE_LIEU
4      TABLE ACCESS BY ROWID    Auteur           a
5*       INDEX UNIQUE SCAN      BTREE_Auteur

(2) valeur_assurance > 50000
(3) lieu_exposition = 'Salle 1'
(5) a.id_auteur = o.id_auteur
```

1.4 Concurrence (4 points)

On considère l'exécution concurrente suivante :

$$H = r_1[x]r_3[y]r_3[x]w_3[x]r_2[x]C_3w_2[x]r_1[x]w_1[y]C_1C_2$$

On suppose que les trois transactions débutent au même moment.

1.4.1 Considérons la dernière lecture $r_1[x]$. Quelle est la valeur de x lue dans, respectivement, les modes suivants:

(a) READ UNCOMMITTED ?

(b) READ COMMITTED ?

(c) REPEATABLE READ ?

Commenter brièvement chaque réponse. (1 point)
Correction :
En (1), on lit la valeur écrite par T_2 (lecture sale) ; en (2) on lit la valeur écrite par T_3 (dernière valeur validée) ; en (3) on lit la valeur existant au début de la transaction (lecture cohérente).

1.4.2 Identifier un nombre suffisant de conflits montrant que cette exécution n'est pas sérialisable. (1 point)
Correction :
Conflits entre $r_1[x]$ et $w_3[x]$ (donc $T_1 \leq T_3$), entre $w_3[x]$ et $r_2[x]$ (donc $T_3 \leq T_2$), entre $r_1[x]$ et $w_2[x]$ (donc $T_1 \leq T_2$), $r_3[y]$ et $w_1[y]$ (donc $T_3 \leq T_1$), etc. Clairement le graphe de $\leq$ est cyclique et l'exécution n'est pas sérialisable.

1.4.3 Appliquer un algorithme de verrouillage à deux phases et donner l'ordonnancement obtenu. (2 points)
Correction :

(a) T_1 et T_3 posent des verrous partagés sur x;

(b) T_3 a posé un verrou partagé sur y;

(c) T_3 est alors bloquée en tentant d'exécuter $w_3[x]$;

(d) T_2 pose également un verrou partagé sur x mais se retrouve bloquée sur $w_2[x]$;

(e) T_1 lit à nouveau x, ce qui est autorisé;

(f) T_1 tente de poser un verrou exclusif sur y, ce qui est impossible à cause du verrou partagé de T_3. **Interblocage**.

Conclusion: aucune écriture n'a pu s'effectuer dans ce cas. Au moment de l'interblocage on a é les opérations suivantes :

$$H' = r_1[x]r_3[y]r_3[x]r_2[x]r_1[x]$$

2 | NFP 107 FOD, 12 avril 2012

2.1 Algèbre relationnelle et SQL (10 points)

Soit la base de données suivante (simplifiée) qui sert à l'évaluation des projets déposés en réponse à un appel d'offres. Chaque projet a un auteur et doit avoir entre 3 et 5 rapporteurs (rapporteur = personne qui prépare un rapport d'évaluation). Une personne peut déposer zéro, un ou plusieurs projets. Une personne peut être rapporteur pour un seul projet. Les conflits d'intérêt doivent être évités : d'abord, une personne ne peut pas être rapporteur pour les projets dont elle est l'auteur ; les autres conflits d'intérêt sont répertoriés dans la table CONFLIT : chaque enregistrement dans cette table correspond à une paire [personne1, personne2] telle que personne1 ne peut pas être rapporteur pour un projet dont l'auteur est personne2.

Personne(id_personne, nom, prénom, date_naissance)
Projet(id_projet, id_auteur, nom_projet, description)
Rapport(id_rapporteur, id_projet, texte_évaluation, date)
Conflit(id_rapporteur, id_auteur)

2.1.1 Pour chaque table, identifier les clés primaires et les clés étrangères. (1 point)
Correction :

- Clés primaires
  Table `Personne : id_personne`
  Table `Projet : id_projet`
  Table `Rapport : id_rapporteur` (vu qu'une personne peut être rapporteur pour un seul projet)
  ou éventuellement `Rapport : (id_rapporteur, id_projet)`
  Table `Conflit : (id_rapporteur, id_auteur)`

- Clés étrangères :
  `Projet[id_auteur]` $\subseteq$ `Personne[id_personne]`
  `Rapport[id_rapporteur]` $\subseteq$ `Personne[id_personne]`
  `Rapport[id_projet]` $\subseteq$ `Projet[id_projet]`
  `Conflit[id_rapporteur]` $\subseteq$ `Personne[id_personne]`
  `Conflit[id_auteur]` $\subseteq$ `Personne[id_personne]`

2.1.2 Donner des commandes SQL permettant de créer les tables **Rapport** et **Conflit**. (1 point)

Correction :

```
CREATE TABLE Rapport (
id_rapporteur INTEGER NOT NULL,
id_projet INTEGER NOT NULL,
texte_évaluation VARCHAR2(2000),
date DATE,
CONSTRAINT Rapport_PK PRIMARY KEY (id_rapporteur),
CONSTRAINT Rapport_id_rapporteurKF FOREIGN KEY (id_rapporteur) REFERENCES Personne
CONSTRAINT Rapport_id_projetKF FOREIGN KEY (id_projet) REFERENCES Projet (id_projet) );
);

CREATE TABLE Conflit (
id_rapporteur INTEGER NOT NULL,
id_auteur INTEGER NOT NULL,
CONSTRAINT Conflit_PK PRIMARY KEY (id\_rapporteur, id_auteur),
CONSTRAINT Conflit_id_rapporteurKF FOREIGN KEY (id_rapporteur) REFERENCES Personne (i
CONSTRAINT Conflit_id_auteurKF FOREIGN KEY (id_auteur) REFERENCES Personne (id_perso
```

2.1.3 Donner les identifiants des projets pour lesquels au moins un rapport a été transmis après le 01-01-2012. Algèbre et SQL (1 point)
Correction :

Algèbre:
$\pi_{id_projet}(\sigma_{date \geq 01-01-2012}(Rapport))$
SQL:

```
SELECT id_projet FROM Rapport
WHERE date > 01−01−2012;
```

2.1.4 Donner le nom de l'auteur qui a le plus de conflits d'intérêt. SQL seulement. (2 points)

Correction :

```
SELECT id_personne, nom, count(∗)
FROM Personne, Conflit
WHERE id_personne = id_rapporteur
GROUP BY id_personne
HAVING count(∗) >= ALL (SELECT count(∗)
                        FROM Conflit
                        GROUP BY id_rapporteur);
```

2.1.5 Donner, pour chaque personne (nom et prénom), le nombre de projets dont elle a été auteur; les résultats doivent être triés par ordre décroissant du nombre de projets. SQL seulement (1 point)
Correction :

```
SELECT id_personne, nom, prénom, count(id_projet) AS nbProjets
FROM Personne, Projet
WHERE id_personne = id_auteur
GROUP BY id_personne
ORDER BY nbProjets DESC;
```

2.1.6 Donner les identifiants des projets qui ont un nombre de rapports insuffisant (moins de 3). SQL seulement. (2 points)

Correction :

SQL :

```
SELECT id_projet, count(id_rapporteur)
FROM Rapport
GROUP BY id_projet
HAVING count(id_rapporteur) < 3;
```

2.1.7 Donner les identifiants des personnes ayant fait un rapport pour chaque projet de Jean DUPONT. Algèbre seulement. (1 point)
Correction :

R_1 (les identifiants des projets de Jean DUPONT) :
$\pi_{id_projet}(Projet \bowtie_{id_auteur=id_personne} (\sigma_{nom='DUPONT' \wedge prnom='Jean'}(Personne)))$.
R_2 (la correction):
$(\pi_{id_rapporteur,id_projet}(Rapport)) \div R_1$

2.2 Indexation (3 points)

On insère successivement les enregistrements suivants dans la table Projet, selon cet ordre :

id_projet	id_auteur	**nom_projet**	description
1	5	AERO	...
2	3	MAXIMIN	...
3	10	NETSET	...
4	7	SYSTEME	...
5	4	STAR	...
6	5	TRECVID	...
7	12	SGAME	...
8	4	INITIAL	...
9	2	METRONOM	...
10	1	VIASAT	...
11	3	HERERA	...
12	5	DOCEND	...
13	8	DIMINUTIS	...
14	9	EXSAT	...
15	10	DATASEC	...
16	2	LEMME	...
17	12	VISTARA	...

On met en place un index sur l'attribut **nom** de cette table.

2.2.1 Construire l'arbre B+ d'ordre **2** correspondant à cet ensemble d'articles, en respectant l'ordre d'arrivée. Donner les étapes de construction intermédiaires importantes (celles qui produisent un changement de la structure de l'arbre) et distinguer sur les schémas les nœuds de natures différentes (nœuds contenant des clés vs. ceux contenant les enregistrements complets ou autre chose). (2 points)

Correction :

La figure 5.1 donne l'arbre B+ d'ordre 2 obtenu. Ici les nœuds internes (en italique) contiennent les clés des enregistrements, alors que les nœuds feuilles (en gras) contiennent les enregistrements complets, ou bien leurs clés plus les pointeurs vers ces enregistrements sur disque.

Figure 2.1: Arbre B+ d'ordre 2.

2.2.2 On recherche les auteurs des projets dont l'initiale du nom se trouve entre 'D' et 'I' (inclus). Combien de pages disque doivent être chargées pour répondre à cette requête en utilisant l'arbre B+ ? Justifier. (1 point)

Correction :

Il s'agit ici d'une requête par intervalle, facilement réalisable avec un arbre B+. On commence par faire une recherche du premier enregistrement dont l'initiale du nom est égale ou immédiatement supérieure à 'H'. On charge le nœud racine 'Nelson' puis son fils gauche puisque 'H' précède 'N'. 'H' se situant entre 'Eames' et 'Jacobsen', on lit ensuite le nœud feuille relatif aux enregistrements 'Gray' et 'Jacobsen'. Jusque là trois nœuds, et donc trois pages, ont été chargés. Dans cette dernière page chargée, 'Jacobsen' est sélectionné car plus grand que 'H' tout en étant plus petit que 'N'. On a trouvé le premier enregistrement répondant à la requête, il suffit d'exploiter le chaînage des feuilles pour trouver les autres enregistrements : on lit ainsi les deux nœuds feuilles suivants, jusqu'à l'enregistrement 'Panton' qui ne rentre par dans l'intervalle recherché. On a trouvé cinq enregistrements répondant à la requête ('Jacobsen', 'Jouin', 'Le Corbusier', 'Nelson' et 'Noguchi'), et lu deux pages disque de nœuds internes, plus trois pages disque de nœuds feuilles. Si les feuilles contiennent les enregistrements complets, alors les prénoms recherchés y figurent et on a fini : **on a lu cinq pages disque en tout**. Sinon, il faut accéder au fichier des données à partir des pointeurs contenus dans les feuilles. Dans ce cas on doit lire au minimum une page disque (cas optimiste où les cinq enregistrements sélectionnés sont tous sur la même page) et au maximum cinq pages (cas pessismiste où ils sont tous répartis sur des pages différentes) : **on aura donc lu entre six et dix pages en tout**.

2.3 Optimisation (3 points)

2.3.1 Donner le plan EXPLAIN obtenu pour la requête suivante (2 points) :

SELECT pe.nom, pe.prenom
FROM Personne pe, Projet pr
WHERE pr.id_auteur = pe.id_personne AND
pr.nom_projet = 'METRONOM' ;

Sachant que nous avons qu'un index BTREE_NOM_PROJET sur le nom du projet (comme créé dans la partie 2), le nom n'est pas forcément unique. Vous ferez apparaître les conditions de la requêtes sous le plan EXPLAIN (comme indiqué dans le cours). Expliquer votre choix.

Correction :

```
0  SELECT STATEMENT
1    NESTED LOOP JOIN
2      TABLE ACCESS BY ROWID    Projet           pr
3*       INDEX RANGE SCAN       BTREE_NOM_PROJET
4*     TABLE ACCESS FULL        Personne         pe

(3) nom_projet = METRONOM
(4) id_auteur = id_personne
```

2.3.2 Sachant que nous ajoutons un index de type B+Tree sur la clé primaire **Personne.id_personne**, modifier le plan EXPLAIN précedent pour y intégrer cet index. Expliquer votre choix (1 point) :

Correction :

```
0  SELECT STATEMENT
1    NESTED LOOP JOIN
2      TABLE ACCESS BY ROWID    Projet           pr
3*       INDEX UNIQUE SCAN      BTREE_NOM_PROJET
4      TABLE ACCESS BY ROWID    Personne         pe
5*       INDEX UNIQUE SCAN      BTREE_ID_PERSONNE

(3) nom_projet = METRONOM
(5) id_auteur = id_personne
```

2.4 Concurrence (4 points)

2.4.1 Indiquez si les exécutions concurrentes suivantes peuvent produire des anomalies (et lesquelles), voire être sérialisable. (1 point)

- $r_1[x]w_1[x]r_2[x]w_2[y]R_1C_2$
- $r_1[x]w_1[x]r_2[y]w_2[y]R_1C_2$
- $r_1[x]r_2[x]r_2[y]w_2[y]r_1[z]R_1C_2$
- $r_1[x]r_2[x]w_2[x]w_1[x]C_1C_2$
- $r_1[x]r_2[x]w_2[x]r_1[y]C_1C_2$
- $r_1[x]w_1[x]r_2[x]w_2[x]C_1C_2$

Correction :

- Lecture sale de T_2 sur T_1.
- Aucun conflit.
- Aucun conflit.
- Risque de mise à jour perdue.
- Exécution sérialisable.
- Idem.

2.4.2 Indiquez si l'exécution suivante est sérialisable. Justifiez votre réponse en donnant les conflits et un graphe de sérialisation. (1 point)

- $r_1[x]r_2[x]w_2[x]C_2r_3[x]r_4[z]w_1[x]w_3[y]w_3[x]C_3w_1[y]w_5[x]w_1[z]C_1w_5[y]r_5[z]C_5C_4$

Correction :

- Conflits cycliques entre T_2 et T_3.

2.4.3 Faire l'exécution précédente avec un protocole de verrouillage à deux phases. Donner le déroulement de l'histoire qui en résulte (avec les verrous), et les interblocages s'ils existent (*deadlock*) ? (2 points)

3 | NFP 107 HTT, 4 Juillet 2012

3.1 Algèbre relationnelle et SQL (10 points)

Voici une partie de la base de données utilisée pour gérer les jeux Olympiques de Londres :
`EPREUVE(ID, nom, recordMonde, recordOlympique)`
`ATHLETE(ID, nom, prenom, sexe, date_naiss, nationalite)`
`CLASSEMENT(`*IDAthlete, IDEpreuve*`, classement, record)`
Le record est une chaîne de caractères contenant la valeur du record et sa mesure (8.45s, 8,47m, 146kg, ...).
Le classement correspond à la place de l'athlète dans l'épreuve (1 = or, 2 = argent, 3 = bronze, ...).

3.1.1 (1 point) Donner le schéma conceptuel entités/associations correspondant à ce schéma relationnel, en précisant si vous avez employé la notation UML ou MERISE.
Correction :
Deux entités Epreuve et Athlete. Une association Classement (avec deux attributs).

3.1.2 (1 point) Répondre aux questions suivantes en argumentant brièvement votre réponse :

- Un athlète peut-il avoir deux classements différents pour la même épreuve ?
  Correction :
  Non, l'association est unique pour une épreuve et un athlète.
- Un athlète peut-il participer à plusieurs épreuves différentes ?
  Correction :
  Oui, rien n'empêche l'association l'appartenance d'un athlète à deux épreuves.
- Un athlète peut-il avoir un record personnel supérieur au record mondial de l'épreuve correspondante ?
  Correction :
  Oui, aucune contrainte n'est visible sur la notion de record personnel.
- Deux athlètes différents peuvent-ils avoir la médaille d'or (classement = 1) pour la même épreuve ?
  Correction :
  Oui, malheureusement, il n'y a pas de contraintes sur le classement. Donc c'est possible.

Pour les questions suivantes, indiquer pour chaque expression donnée une requête en algèbre et/ou en SQL :

3.1.3 (1 point - Algèbre + SQL) Le record des athlètes qui ont eu une médaille d'or ?
Correction :
$\pi_{record}(\sigma_{classement=1}(CLASSEMENT))$

SELECT record
FROM CLASSEMENT
WHERE classement = 1;

3.1.4 (1 point - Algèbre + SQL) Le nom des épreuves pour lesquelles le record du Monde est égal à son record Olympique ?
Correction :
$\pi_{nom}(\sigma_{recordMonde=recordOlympique}(EPREUVE))$

SELECT nom
FROM EPREUVE
WHERE recordMonde=recordOlympique;

3.1.5 (1 point - Algèbre + SQL) Liste des pays distincts ayant eu une médaille (or, argent ou bronze) ?
Correction :
$\pi_{nationalite}(\sigma_{classement<=3}(CLASSEMENT) \bowtie_{IDAthlete=ID} ATHLETE)$

SELECT DISTINCT nationalite
FROM CLASSEMENT, ATHLETE
WHERE IDAthlete = ID **AND** CLASSEMENT **BETWEEN** 1 **AND** 3 ;

3.1.6 (1.5 point - SQL) Nombre de médailles par pays, classés par ordre décroissant de nombre de médailles ?
Correction :

SELECT nationalite, **count**($*$) **AS** NB
FROM CLASSEMENT, ATHLETE
WHERE IDAthlete = ID **AND** CLASSEMENT **BETWEEN** 1 **AND** 3
GROUP BY nationalite
ORDER BY NB **DESC** ;

3.1.7 (1.5 point - Algèbre + SQL) Le nom des athlètes ayant eu une médaille à chaque épreuve qu'il a effectué ?
Correction :
$\pi_{nom}(\pi_{id,nom}(ATHLETE) - \pi_{id,nom}(ATHLETE \bowtie_{IDAthlete=ID} \sigma_{classement>3} CLASSEMENT))$

SELECT nom
FROM ATHLETE
WHERE ID **NOT IN** (**SELECT** IDAthlete **FROM** CLASSEMENT **WHERE** CLASSEMENT > 3) ;

3.1.8 (2 points - SQL) Le nom du pays ayant eu le plus de médailles, ainsi que le nombre de ces médailles ?
Correction :
$\pi_{nom,prenom}(\rho_{id \to id_aud}(AUDITEUR) \bowtie (INSCRIPTION \div \rho_{id \to id_ue}(\pi_{id}(UE))))$

SELECT nationalite
FROM ATHLETE, CLASSEMENT
WHERE IDAthlete = ID **AND** classement < 4
GROUP BY nationalite
HAVING COUNT(∗) = (**SELECT MAX**(**COUNT**(∗)
FROM ATHLETE, CLASSEMENT
WHERE IDAthlete = ID **AND** classement < 4
GROUP BY nationalite);

3.2 Index (2 points)

3.2.1 (2 points) Créer un index B+Tree d'ordre 2 sur la nationalité des Athlètes avec les valeurs ci-dessous. Toute valeur en doublon apparaîtra plusieurs fois. Vous montrerez chaque étape d'éclatement de l'arbre.

fr, en, us, au, it, ch, br, fr, us, us, de, ru, ru,

3.3 Optimisation (3 points)

3.3.1 (3 points) Pour la requête ci-dessous, dire ce quelle produit comme résultat et donner un plan d'exécution (EXPLAIN et arborescent) sachant qu'il existe un index sur l'attribut `classement` de la table `CLASSEMENT`, qu'il n'y a pas d'autres index et que les deux tables sont de tailles moyennes.

SELECT NOM, PRENOM
FROM Athlete A, CLASSEMENT C
WHERE A.ID=C.IDAthlete
AND C.classement = 1
AND A.nationalite = 'Fr'

Correction :
Cette requête donne les noms et prénoms des Athlètes Français classés premiers dans une épreuve.

```
0  SELECT STATEMENT
1*   NESTED LOOPS
2      TABLE ACCESS BY INDEX ROWID   Classement
3*       INDEX RANGE SCAN            IDX_CLASSEMENT
4*     TABLE ACCESS FULL             Athlete

(1) A.ID=C.ID_Athlete
```

```
(3) CLASSEMENT = 1
(4) a.nationalite = 'Fr'
```

3.4 Concurrence (5 points)

Soit les deux programmes de locations suivants[1] :

Programme P1 :

- ```
  Nouveau_record_personnel (IDAthlete, IDEpreuve, record)
    var tmp
  begin
    Start    //Lecture puis mise à jour du record de l'athlète pour une épreuve
    tmp = Read (Classement, IDAthlete, IDEpreuve)
    Write (Classement, IDAthlete, IDEpreuve, record)
    //Lecture de l'épreuve correspondante pour voir s'il faut changer le record
    tmp = Read (Epreuve, IDEpreuve)
    if (tmp.recordOlympique < record)                  //Mise à jour du record
      Write (Epreuve, IDEpreuve, record)
    end if
    Commit
  end
  ```

Programme P2 :

- ```
  ModifClassement (IDEpreuve, IDAthlete, classement)
    var tmp
  begin
    Start                              //Mise à jour du classement d'un athlete
    Write (Classement, IDEpreuve, IDAthlete, classement)
    //Verification si un autre athlete a déjà ce classement (il en existe au plus un)
    tmp = Read (Classement, IDEpreuve, IDAthlete)
    if (tmp == null)       //S'il n'y en a pas (fin de chaine), on finalise la transaction
      Commit
    else             //Sinon, il faut décaler cet ancien athlète d'un cran, par appel récursif
      ModifClassement (Classement, IDEpreuve, temp.IDAthlete, temp.classement + 1)
    end if
  end
  ```

Soit l'exécution concurrente suivante (les verrous se posent au niveau du n-uplet) :
H: $w_1[x]r_2[y]r_3[z]w_2[y]r_1[y]w_1[y]r_2[e]w_3[z]r_1[z]w_2[e]c_2r_3[e]w_1[z]r_1[t]w_3[e]c_3c_1$

3.4.1 (1 points) Identifier pour chaque transaction à quel programme (P1 ou P2) il peut correspondre.

Correction :
T1 : programme P2 $\Rightarrow w_1[x]r_1[y]w_1[y]r_1[z]w_1[z]r_1[t]c_1$
T2 : programme P1 $\Rightarrow r_2[y]w_2[y]r_2[e]w_2[e]c_2$
T3 : programme P1 $\Rightarrow r_3[z]w_3[z]r_3[e]w_3[e]c_3$

[1] Chaque opération de lecture donne en premier paramètre la table sélectionnée, puis l'identifiant correspondant, et enfin dans le cas des écritures la valeur modifiée.

3.4.2 (1 point) Identifier les conflits et donner le graphe de sérialisation. Cette histoire est-elle sérialisable ?

Correction :

Conflits : $r_2[y] - w_1[y]$, $w_2[y] - r_1[y]$, $w_2[y] - w_1[y]$, $r_2[e] - w_3[e]$, $w_3[z] - r_1[z]$, $w_2[e] - r_3[e]$, $w_2[e] - w_3[e]$

Il n'y a pas de cycles, il est sérialisable.

3.4.3 (2 points) Cette histoire est-elle recouvrable? garantie-t'elle l'annulation en cascade ? Est-elle stricte ? Expliquer.

Correction :

- H est Recouvrable :
  - $w_3[z] - r_1[z]$: c_1 après le c_3
  - $w_2[e] - r_3[e]$: c_3 après le c_2
- H ne garanti pas les annulations en cascade :
  - $w_3[z] - r_1[z]$: c_3 après la lecture
  - $w_2[e] - r_3[e]$: c_2 avant la lecture
- H n'est pas strict car ne garanti pas l'annulation en cascade.

3.4.4 (1 point) Un crash du système survient juste après c_2. Etant donné que l'opération $w_1[x]$ donnait pour nouveau classement à x la valeur 3. Quel est le classement de z <u>après</u> la reprise du système ?

Correction :

Le programme P1 décale tous les classements de 1 qui sont égaux au premier. Ainsi, on peut en déduire que y passe de 3 à 4, z de 4 à 5. $w_3[z]$ modifie le record mais pas le classement donc pas d'interférence.

Ainsi, lors du crash seule la transaction 2 est valide, et il faut annuler toutes les modifications de T1 et T3. On peut alors en déduire que nous revenons dans l'état initial au niveau du classement puisque seul T1 modifiait le classement.

Donc, z vaut 4 après la reprise.

4 | NFP 107 HTT, 12 septembre 2012

4.1 Algèbre relationnelle et SQL (9 points)

Voici une partie de la base de données utilisée par un site de vente aux enchères :

```
ARTICLE(idArticle, nom, description)
PERSONNE(idPersonne, nom, prenom, adresse, compte, noteVendeur, noteAcheteu
MISEENVENTE(idArticle, idVendeur, prixMiseEnVente, dateMiseEnVente)
VENTE(idArticle, idVendeur, idAcheteur, prixVendu, dateVente)
```

4.1.1 (1 point) Pour chaque table, identifier les clés primaires et les clés étrangères.

4.1.2 (2 points) Donner le schéma conceptuel entités/associations correspondant à ce schéma relationnel, en précisant si vous avez employé la notation UML ou MERISE.

Pour les questions suivantes, indiquer pour chaque expression donnée une requête en algèbre et/ou en SQL :

4.1.3 (1 point - Algèbre + SQL) Quels sont les articles dont le prix de mise en vente est compris entre 100 et 200 euros?

4.1.4 (1,5 points - Algèbre + SQL) Quels vendeurs n'ont vendu aucun des articles qu'ils ont mis en vente?

4.1.5 (1 point - SQL) Pour les vendeurs pour lesquels le montant total des ventes est supérieur à 10000 euros, indiquer idPersonne, nom, prenom et montant total des ventes.

4.1.6 (1 points - SQL) Quel vendeur a mis en vente le plus d'articles?

4.1.7 (1,5 points - Algèbre + SQL) Quels acheteurs ont acheté au moins un article de chaque vendeur?

4.2 Index (3 points)

On met en place un index sur l'attribut `prixMiseEnVente` de la table `ARTICLE`. On considère que les prix de mise en vente des articles insérés dans cette table sont, dans l'ordre d'insertion: 28, 55, 210, 120, 33, 34, 65, 82, 2, 29, 10, 80.

4.2.1 (2 points) Construire l'arbre B+ d'ordre 2 correspondant à cet ensemble de valeurs, en respectant l'ordre d'arrivée. Donner les étapes de construction intermédiaires importantes (celles qui produisent un changement de la structure de l'arbre) et distinguer sur les schémas les noeuds de natures différentes (noeuds contenant des clés vs. ceux contenant les enregistrements complets ou autre chose).

4.2.2 (1 point) On recherche les articles dont le prix de mise en vente est entre 50 et 150 euros (inclus). Combien de pages disque doivent être chargées pour répondre à cette requête en utilisant l'arbre B+ ? Justifier.

4.3 Optimisation (4 points)

4.3.1 (4 points) Pour la requête ci-dessous, exprimer en une phrase ce qu'elle produit comme résultat et donner un plan d'exécution (EXPLAIN et arborescent) sachant qu'il existe un index sur l'attribut `idPersonne` de la table `Personne`, qu'il n'y a pas d'autres index, que la table `MISEENVENTE` est de taille moyenne et la table `VENTE` est plus petite que la précédente.

SELECT NOM, PRENOM
FROM Personne P, MISEENVENTE M, VENTE V
WHERE P.idPersonne = V.idAcheteur **AND**
V.idArticle = M.idArticle **AND**
prixMiseEnVente < prixVendu / 10 ;

Correction :
Cette requête donne les noms et prénoms des acheteurs qui ont acheté des articles 10 fois plus chers que leur prix de mise en vente.

```
0  SELECT STATEMENT
1    NESTED LOOPS
2*     NESTED LOOPS
3        TABLE ACCESS FULL           VENTE
4        TABLE ACCESS FULL           MISEENVENTE
5      TABLE ACCESS BY INDEX ROWID   PERSONNE
6*       INDEX RANGE SCAN            IDX_PERSONNE

(2) V.idArticle = M.idArticle AND prixMiseEnVente < prixVendu / 10
(6) P.idPersonne = V.idAcheteur
```

4.4 Concurrence (4 points)

Soit l'exécution concurrente suivante (les verrous se posent au niveau du n-uplet) :
H: $r_2[y]r_1[x]r_3[z]w_2[y]r_1[y]w_1[y]w_3[z]r_2[t]r_1[z]w_2[t]c_2w_1[z]r_3[x]r_1[t]w_3[t]c_3c_1$

4.4.1 (1 point) Identifier pour chaque donnée (x,y,z,t), les conflits associés et donner le graphe de sérialisation. Cette histoire est-elle sérialisable ?

Correction :
Conflits : $r_1[x] - r_3[x]$ $r_2[y] - w_1[y]$, $w_2[y] - r_1[y]$, $w_2[y] - w_1[y]$,
$r_2[t] - w_3[t]$, $w_2[t] - r_3[t]$, $w_2[t] - r_1[t]$, $w_2[t] - w_3[t]$, $r_1[t] - w_3[t]$
$w_3[z] - r_1[z]$
Il y a un cycle, l'histoire n'est pas sérialisable.

4.4.2 (2 points) Trouver l'exécution produite par verrouillage à deux phases simple si les verrous d'une transaction sont toujours relâchés après sa validation ou son annulation.
Correction :
$r_2[y]w1[x]r_3[z]w_2[y]w_3[z]r_2[t]w_2[t]c_2r_1[y]r_3[x]w_3[t]c_3r_1[z]w_1[z]r_1[t]c_1$
Exécution sans problèmes.

4.4.3 (1 point) On pose un verrou exclusif avec la commande FOR UPDATE pour les instructions de lecture de la transaction T1. Le résultat est-il différent ? Que se passe-t-il ?
Correction :
Cette fois-ci, cela fera bloquer la transaction T3 sur $r_3[x]$, ce qui donnera alors un interblocage.

5 | NFP 107 FOD, 9 février 2013

5.1 Algèbre relationnelle et SQL (10 points)

Soit la base de données suivante (simplifiée) de gestion du 22ème Open International d'échecs de la ville de Cnamville :

```
  PARTICIPANT (idParticipant, nom, prénom, age, nationalité);
PARTIE (idBlanc, idNoir, ouverture, résultat);
classementELO (idJoueur, coeffELO);
```

Les attributs idBlanc et idNoir représentent les identifiants des joueurs qui jouent avec les blancs et, respectivement, les noirs. Le coefficient ELO représente la force de jeu du joueur, donnée par ses résultats dans le passé. Le champ résultat signifie le résultat d'une partie : 1 si le blanc gagne, -1 si le noir gagne et 0 en cas de nullité. A ce tournoi on ne peut avoir que des joueurs de force plus grande de 2600 ELO (niveau grand maître).

5.1.1 (1 point) Pour chaque table, identifier les clés primaires et les clés étrangères. (1 point)
Correction :
PARTICIPANT : idParticipant clé primaire
PARTIE : couple (idBlanc, idNoir) clé primaire ; idBlanc et idNoir clés étrangères.
classementELO : idJoueur clé primaire et clé étrangère

5.1.2 (1 point) Donner la commande SQL permettant de créer la table PARTIE. (1 point)
Correction :

CREATE TABLE PARTIE (
idBlanc **INTEGER NOT NULL**,
idNoir **INTEGER NOT NULL**,
ouverture **VARCHAR2**(40),
résultat **INTEGER**,
CONSTRAINT partie_pk **PRIMARY KEY** (idBlanc, idNoir),
CONSTRAINT partie_blanc_fk **FOREIGN KEY** idBlanc
REFERENCES PARTICIPANT (idParticipant),
CONSTRAINT partie_noir_fk **FOREIGN KEY** idNoir
REFERENCES PARTICIPANT (idParticipant))

Pour les questions suivantes, indiquer pour chaque expression donnée une requête en algèbre et/ou en SQL :

5.1.3 Donnez les résultats des parties dont l'ouverture est la "Défense française". Algèbre et SQL. (1 point)

Correction :
Algèbre : (0.5 points)
$\pi_{resultat}(\sigma_{ouverture='DefenseFrancaise'}(PARTIE))$
SQL (0.5 points):

SELECT résultat
FROM PARTIE
WHERE ouverture = ‘‘Défense française’’;

5.1.4 Donnez les noms de joueurs de force de jeu de moins de 2700 ELO qui ont gagné avec les blancs contre la “Défense française” (gagné avec les blancs une partie dont la l'ouverture est la “Défense française”). Algèbre et SQL. (2 points)
Correction :
Algèbre (1 point):

SQL (1 point):

SELECT nom
FROM PARTICIPANT, PARTIE, classementELO
WHERE idBlanc = idJoueur **AND** idBlanc = idParticipant
AND ouverture = 'Defense française'
AND coeffELO $<$ 2700
AND résultat = 1 ;

5.1.5 Quelle est la force de jeu moyenne (coeffELO) des joueurs français qui ont moins de 30 ans ? SQL seulement. (1 point)
Correction :

SELECT AVG(coeffELO)
FROM classementELO, PARTICIPANTS
WHERE idParticipant = idJoueur **AND** nationalité = 'français' **AND** age < 30 ;

5.1.6 Donnez les noms des participants qui n'ont jamais gagné avec les blancs contre Garry Kasparov. Ces participants n'ont pas forcément joué contre Kasparov. SQL seulement. (2 points)
Correction :

SELECT NOM **FROM** PARTICIPANT, PARTIE
WHERE idBlanc = idParticipant **AND** idBlanc **not in**
(**SELECT** idBlanc **FROM** PARTICIPANT, PARTIE
WHERE idNoir = idParticipant **AND** nom = 'Kasparov' **AND**
AND prénom = 'Garry' **AND** résultat = 1)

La partie imbriquée représente les joueurs qui ont gagné au moins une partie avec les blancs contre Garry Kasparov.

5.1.7 Donner la nationalité qui a le plus de participants ayant gagné avec les noirs. SQL seulement. (2 points)
Correction :
SELECT nationalité FROM PARTICIPANT, PARTIE WHERE idParticipant = idNoir AND résultat = -1 GROUP BY nationalité HAVING COUNT(*) = (SELECT MAX(COUNT(*)) FROM PARTICIPANT, PARTIE WHERE idParticipant = idNoir AND résultat = -1 GROUP BY nationalité)

5.2 Index (3 points)

Créer l'index B+Tree d'ordre 2 sur l'attribut CoeffELO avec les valeurs suivantes (s'il y a des doublons, on fera apparaître chaque valeur) :
2800, 2900, 3000, 2600, 2700, 2950, 2820, 2850, 2870, 2910, 2880, 3100, 3200, 2975, 2990
Correction :

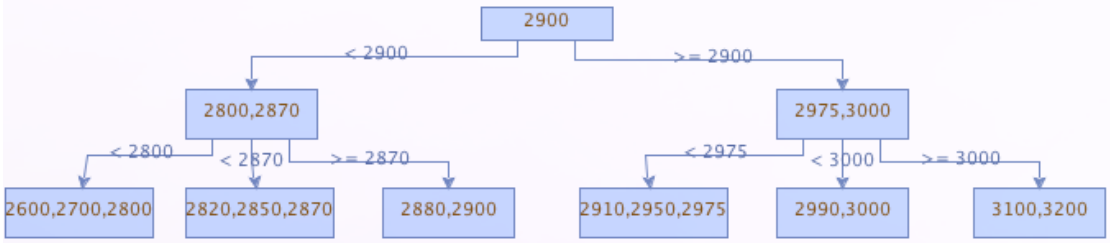


Figure 5.1: Arbre B+ d'ordre 2.

5.3 Optimisation (4 points)

5.3.1 (1 point) Pour la requête ci-dessous, dire ce quelle produit comme résultat (sous forme de phrase)

SELECT P1.NOM
FROM PARTICIPANT P1, classementELO c1, classementELO c2, PARTIE PA
WHERE P1.idParticipant = c1.idJoueur **AND**
PA.idBlanc = P1.idParticipant **AND**
PA.idNoir = c2.idJoueur **AND**
(c2.coeffELO − c1.coeffELO) > 300 **AND**
résultat >= 0;

Correction :
Cette requête donne les noms des joueurs qui ont gagné ou fait nulle en tant que blanc contre un joueur ayant 300 ELO de plus que lui.

5.3.2 (3 points) Donner un plan d'exécution (EXPLAIN ou arborescent) pour cette requête, sachant qu'il existe un index sur l'attribut `idJoueur` de la table `classementELO`, et sur l'attribut `idParticipant` de la table `PARTICIPANT`. Toutes les tables sont de petite taille.
Correction :
3 jointures. On peut utiliser 2 jointures par Boucle imbriquée avec index.

5.4 Concurrence (3 points)

Soit l'exécution concurrente suivante (les verrous se posent au niveau du n-uplet) :

$$H : r_1[x]r_2[x]r_3[y]w_1[x]w_1[z]r_3[z]C_1w_2[z]C_2w_3[x]C_3$$

5.4.1 (1 point) Cette histoire est-elle sérialisable ? Démontrer.
Correction :
$r_2[x] - w_3[x]$ et $r_3[z] - w_2[z]$. Suffisant pour montrer un cycle.

5.4.2 (2 points) Trouver l'exécution produite par verrouillage à deux phases simple si les verrous d'une transaction sont toujours relâchés après sa validation ou son annulation. Si un interblocage survient, on supprime la plus ancienne transaction (sans la réinsérer) et on continue.
Correction :
$H' : r_1[x]r_2[x]r_3[y]r_3[z]$
Interblocage, suppression de T1. De nouveau interblocage, suppression de T2. Execution de C_3
$H'' : r_3[y]r_3[z]w_3[x]c_3$

6 | NFP 107 FOD, 16 avril 2013

Base de Données :
Soit la base de données suivante (simplifiée) de gestion du zoo de Cnamville :

```
ESPECE (espèce, carnivore, superficieNecessaire)
ANIMAL(nom, espèce, age, annéeAccueil)
CAGE (zoneParc, espèce, nombreAnimaux, annéeInstallation, superficieCage)
ZONE (nomZone, description, superficieZone, nomResponsable)
```

Dans chaque table, les attributs soulignés constituent la clé primaire de la table. L'attribut *carnivore* vaut vrai si l'animal est carnivore, faux s'il est végétarien et l'attribut *superficieNecessaire* représente la surface dont un représentant (animal) de l'espèce a besoin pour vivre correctement. Chaque cage peut accueillir plusieurs animaux de la même espèce. L'attribut *zoneParc* représente le nom de la zone ou la cage est installée, et l'attribut *espèce* représente partout le nom de l'espèce dont l'animal fait partie.

6.1 Algèbre relationnelle et SQL (10 points)

6.1.1 Pour chaque table, identifier les clés étrangères (1 point).
Correction :
ESPECE : pas de clé étrangère
ANIMAL : espèce $\rightarrow$ ESPECE(espèce)
CAGE : zoneParc $\rightarrow$ ZONE(nomZone), espèce $\rightarrow$ ESPECE(espèce)
ZONE : pas de clé étrangère

6.1.2 (Algèbre et SQL) Donnez les superficies nécessaires pour les animaux accueillis avant 2005 (1 point).
Correction :
Algèbre:
$\pi_{superficieNecessaire}$(ANIMAL $\bowtie$ ESPECE)
SQL:
SELECT DISTINCT superficieNecessaire FROM ANIMAL a , ESPECE e
WHERE a.espèce = e.espèce AND annéeAccuei $<$ 2005;

6.1.3 (Algebre et SQL) Donnez les espèces carnivores dont tous les animaux sont âgés de plus de 10 ans (2 points).
Correction :
Algèbre:
π_{espece}(ANIMAL)$-\pi_{espece}(\sigma_{age<=10 \wedge carnivore=TRUE}$(ANIMAL $\bowtie$ ESPECE))
SQL:
SELECT espèce FROM ANIMAL

WHERE espèce NOT IN
_ (SELECT DISTINCT espèce FROM ANIMAL a, ESPECE e
_ WHERE a.espèce = e.espèce AND age <= 10 AND carnivore = TRUE);

6.1.4 (Algebre et SQL) Qui sont les responsables des zones ou on trouve des animaux de l'espèce "Panda géant" âgés de moins de deux ans ? (2 points)
Correction :
Algèbre:
$\pi_{nomResponsable}(\sigma_{espece='Panda\ geant' \wedge age<2}((\text{ANIMAL} \bowtie \text{CAGE}) \bowtie_{zoneParc=nomZone} \text{ZONE}))$
SQL:
SELECT nomResponsable
FROM ANIMAL a, CAGE c, ZONE z
WHERE z.nomZone = c.zoneParc and a.espece = c.espece AND a.age < 2 AND a.espece = 'Panda géant';

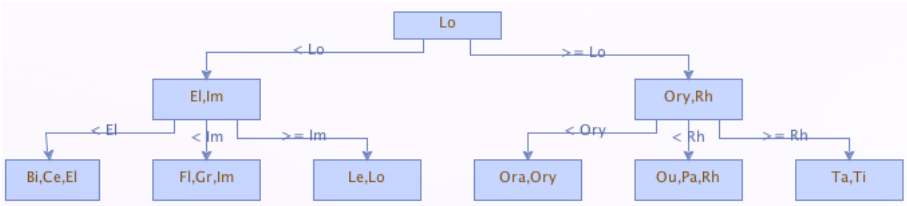
6.1.5 (Algebre et SQL) Dans quelles zones on ne trouve que des animaux carnivores ? (2 points)
Correction :
Algèbre:
$\pi_{nomZone}(\text{ZONE}) - \pi_{nomZone}(\sigma_{carnivore=FALSE}((\text{ESPECE} \bowtie \text{CAGE}) \bowtie_{zoneParc=nomZone} \text{ZONE}))$
SQL:
SELECT nomZone from ZONE WHERE nomZone NOT IN
_ (SELECT nomZone FROM ZONE z, CAGE C, ESPECE e
_ WHERE z.nomZone = c.zoneParc and e.espece = z.espece and e.carnivore = FALSE)

6.1.6 (SQL seulement) Donnez les zones qui accueillent le plus d'animaux ? (2 points)
Correction :
SQL:
SELECT nomZone
FROM WHERE ZONE, CAGE
WHERE nomZone = zoneParc
GROUP BY zoneParc
HAVING SUM(nombreAnimaux) =
_ (SELECT MAX(SUM(nombreAnimaux))
_ FROM ZONE, CAGE
_ WHERE nomZone = zoneParc
_ GROUP BY nomZone))

6.2 Index (2 points)

Créer un arbre B+Tree d'ordre 2 sur les espèces suivantes :
Oryx, Tamarin, Orang-Outan, Elephant, Rhinocéros, Lemur, Loutre, Bison, Ouistitis, Cercopithèque, Grand Koudou, Impala, Tigre, Flamant noir, Panthère rose

Correction :



6.3 Optimisation (4 points)

Donner le plan EXPLAIN et l'arbre d'exécution pour la requête ci-dessous : (4 points)

SELECT nomResponsable
FROM ANIMAL a, CAGE c, **ZONE** z
WHERE z.nomZone = c.zoneParc **AND**
a.espece = c.espece **AND**
a.age < 2 **AND**
a.espece = 'Panda géant';

Sachant que les tables `Animal` et `Cage` sont petites, et que la table `Zone` est assez grande avec un index sur la clé primaire.
Correction :

```
0  SELECT STATEMENT
1    NESTED LOOPS
2*     NESTED LOOPS
3*       TABLE ACCESS FULL           ANIMAL
4        TABLE ACCESS FULL           CAGE
5      TABLE ACCESS BY INDEX ROWID   ZONE
6*       INDEX UNIQUE SCAN           PK_ZONE

(2) a.espece = c.espece
(3) a.age < 2 AND a.espece = 'Panda géant';
(6) z.nomZone = c.zoneParc
```

6.4 Concurrence (4 points)

Soit l'histoire suivante :
H = $r_1[x]\ r_2[x]\ w_3[y]\ r_2[y]\ w_2[y]\ r_3[z]\ w_1[y]\ c_3\ w_1[z]\ w_2[z]\ c_2\ c_1$

6.4.1 H est-elle sérialisable ? Recouvrable ? Evite-t-elle les annulations en cascade ? Est-elle stricte ? `(2 points)`

Correction :

$T_3 \rightarrow T_1, T_3 \rightarrow T_1, T_2 \rightarrow T_1$. Il n'y a pas de cycle, dont H est sérialisable.

Liste des conflits :

sur x aucun conflit ;

sur y $w_3[y]r_2[y]$ $w_3[y]w_2[y]$ $r_2[y]w_1[y]$ $w_2[y]w_1[y]$;

sur z $w_3[z]w_1[z]$;

L'écriture $w_3[y]$ est validée en c_3 avant que les transactions T_1 et T_2 ne valident. De même pour $w_2[y]$. Donc H est recouvrable. Par contre, les lectures sales se font avant le commit c_1, donc elle ne garantie pas l'annulation en cascade, et de fait pas stricte.

6.4.2 Donner l'histoire finale par application de l'algorithme de verrouillage à deux phases. Donner le détail du déroulement de l'algorithme. On suppose que les verrous sont relâchés immédiatement après les *commit*. `(2 points)`

Correction :

1	$r_1[x]$	$VL_1[x]$
2	$r_2[x]$	$VL_2[x]$
3	$w_3[y]$	$VE_3[y]$
4		T_2 bloqué sur $r_2[y]$
5	$r_3[z]$	$VL_3[z]$
6		T_1 bloqué sur $w_1[y]$
7	c_3	$VE_3[y]$ et $VL_3[z]$ relâchés
8	$r_2[y]$	$VL_2[y]$
9	$w_2[y]$	$VE_2[y]$
10		T_1 bloqué sur $w_1[y]$
11	$w_2[z]$	$VE_2[z]$
12	c_2	$VL_2[x]$, $VE_2[y]$ et $VE_2[z]$ relâchés
13	$w_1[y]$	$VE_1[y]$
14	$w_1[z]$	$VE_3[z]$
15	c_1	$VL_1[x]$, $VE_1[y]$ et $VE_3[z]$ relâchés

H' = $r_1[x]$ $r_2[x]$ $w_3[y]$ $r_3[z]$ c_3 $r_2[y]$ $w_2[y]$ $w_2[z]$ c_2 $w_1[y]$ $w_1[z]$ c_1

7 | NFP 107 HTT, Juin 2013

7.1 Algèbre relationnelle et SQL (10 points)

Soit l'extrait suivant de la base de données qui sert à la gestion d'un site web de diffusion de la musique en *streaming*:

```
  ABONNÉ(id, nom, prénom, type_abonnement, date_début)
ARTISTE(id, nom, prénom, nationalité)
ALBUM(id, nom, id_artiste, année_sortie)
CHANSON(id, nom, id_album)
ÉCOUTÉ(id_abonné, id_chanson, date, téléchargé)
```

L'abonnement est sans engagement: seulement la date de début est nécessaire pour un abonnement en cours. Le type d'un abonnement peut être: 'free' (financé par la publicité), 'normal' et 'premium' (donne le droit de télécharger la musique en local pour une écoute hors connexion). L'attribut 'téléchargé' vaut vrai ou faux. Un abonné peut avoir écouté une chanson plusieurs fois (à des dates différentes).

7.1.1 Pour la table ÉCOUTÉ, identifiez les clés primaires et étrangères. (0,5 points) Clé primaire: (id_abonné, id_chanson, date)
Clés étrangères: id_abonné, id_chanson

7.1.2 Donnez la commande SQL pour créer la table ÉCOUTÉ. (0,5 points)

```
CREATE TABLE ÉCOUTÉ(
    id_abonné NUMBER(10),
    id_chanson NUMBER(10),
    date DATE,
    téléchargé BOOLEAN,
    CONSTRAINT pk_ecoute PRIMARY_KEY (id_abonné, id_chanson, date)
    CONSTRAINT fk_ecoute FOREIGN_KEY (id_abonné) REFERENCES ABONNÉ(id),
    CONSTRAINT fk_ecoute FOREIGN_KEY (id_chanson) REFERENCES CHANSON(id
```

7.1.3 (Algèbre et SQL) Donnez les noms des abonnés qui ont souscrit un abonnement 'premium' après le 01/01/2013. (1 point)

$\pi_{nom}(\sigma_{type_abonnement='premium'\ AND\ date>='01/02/2013'}(ABONNE))$

```
SELECT nom FROM ABONNE
WHERE type_abonnement = 'premium' AND date >= '01/02/2013'
```

7.1.4 (SQL seulement) Donnez le nombre d'albums sortis en 2013. (1 point)

```
SELECT COUNT(*)
FROM ALBUM
WHERE année_sortie = 2013
```

7.1.5 (Algèbre et SQL) Donnez les noms des artistes français dont des chansons ont été téléchargées par les abonnés le mois de mars 2013. (2 points)

$$\pi_{nom}(\sigma_{telecharge=TRUE\ AND\ date>='01/03/2013'\ AND\ date<='31/01/2013'}(((ECOUTE \underset{id_chanson=id}{\bowtie} CHANSON) \underset{id_album=id}{\bowtie} ALBUM) \underset{id_artiste=id}{\bowtie} ARTISTE))$$

```
SELECT a.nom
FROM ECOUTE e, CHANSON c, ALBUM al, ARTISTE ar
WHERE e.id_chanson = c.id AND c.id_album = al.id AND al.id_artiste = ar
      AND nationalité = 'française' AND téléchargé = TRUE
      AND date >= '01/03/2013' AND date <= '31/01/2013'
```

7.1.6 (SQL seulement) Donnez les noms des chansons les plus écoutées. (2 points)

```
SELECT nom
FROM ECOUTE, CHANSON
WHERE id_chanson = id
GROUP BY id_chanson
HAVING COUNT(id_chanson) =  (SELECT MAX(COUNT(*))
                             FROM CHANSON
                             GROUP BY id_chanson)
```

7.1.7 (Algèbre seulement) Donnez les noms des abonnés qui ont écouté toutes les chansons sorties en 2013. (1 point)

Les abonnés et les chansons qu'ils ont écoutées :

$A = \pi_{id_abonne,nom,id_chanson}(ECOUTE \underset{id_abonne=id}{\bowtie} ABONNE)$

Toutes les chansons sortis en 2013 :

$B = \pi_{id_chanson}(\rho_{id \to id_chanson}(CHANSON) \underset{id_album=id}{\bowtie} \sigma_{annee=2013}(ALBUM))$

Le résultat est la division entre les deux, c'est-à-dire les abonnés pour lesquels il n'y a pas des chansons de 2013 qu'ils n'ont pas écoutées :

$R = \pi_{nom}(A \div B)$

7.1.8 (Algebre et SQL) Donnez les id des artistes qui n'ont jamais été écoutés (2 points)

Algèbre: tous les artistes **moins** ceux qui ont été écoutés :

$\pi_{id}(ARTISTE) - \pi_{id_artiste}(ECOUTE \bowtie (\rho_{id \to id_chanson, nom \to nom_chanson}(CHANSON) \bowtie$

$\rho_{id \to id_album, nom \to nom_album}(ALBUM)))$

```
SELECT id FROM ARTISTE
WHERE id NOT IN (SELECT a.id_artiste
                 FROM ECOUTE e, CHANSON c, ALBUM a
                 WHERE e.id_chanson = c.id AND c.id_album = a.id)
```

7.2 Indexation (3 points)

On insère successivement les enregistrements suivants dans la table `ARTISTE`, selon cet ordre :

id	**nom**	prénom	nationalité
1	Moustaki	Georges	française
2	Modja	Inna	malienne
3	LeForestier	Maxime	française
4	Vian	Boris	française
5	DePalmas	Gérald	française
6	June	Valérie	américaine
7	Higelin	Jacques	française
8	Berger	Michel	française
9	Goldman	Jean-Jacques	française
10	Mitchell	Eddy	française
11	Katerine	Philippe	française
12	Estefan	Gloria	cubaine
13	Marley	Bob	jamaicaine
14	Azrié	Abed	française

On met en place un index sur l'attribut **nom** de cette table.

7.2.1 Construire l'arbre B+ d'ordre **2** correspondant à cet ensemble d'articles, en respectant l'ordre d'arrivée. Donner les étapes de construction intermédiaires importantes (celles qui produisent un changement de la structure de l'arbre). (2 points)

NB pour la suite : les feuilles de l'arbre B+ "pointent" vers le fichier de données n'apparaissant pas ici.

Insertion de Moustaki, Modja, LeForestier et Vian :

bplus=[rectangle split, rectangle split horizontal,rectangle split ignore empty parts,draw,minimum height=0.6cm] every node=[bplus] level 1=[sibling distance=60mm] level 2=[sibling distance=15mm] LeForestier two Modja three Moustaki four Vian ;

Insertion de DePalmas

bplus=[rectangle split, rectangle split horizontal,rectangle split ignore empty parts,draw,minimum height=0.6cm] every node=[bplus] level 1=[sibling distance=60mm] level 2=[sibling distance=5cm] Modja [->] child[sibling distance=25mm] node (dp) DePalmas two LeForestier three Modja child[sibling distance=25mm, right=of dp] node (mo) Moustaki two Vian three ;

[thick,>=stealth,->] (dp)–+(mo);

Insertion de June

bplus=[rectangle split, rectangle split horizontal,rectangle split ignore empty parts,draw,minimum height=0.6cm] every node=[bplus] level 1=[sibling distance=60mm] level 2=[sibling distance=5cm] Modja [->] child[sibling distance=25mm] node (dp) DePalmas two June three LeForestier four Modja child[sibling distance=60mm] node (mo) Moustaki two Vian three ;

[thick,>=stealth,->] (dp)–+(mo);

Insertion de Higelin

bplus=[rectangle split, rectangle split horizontal,rectangle split ignore empty parts,draw,minimum height=0.6cm] every node=[bplus] level 1=[sibling distance=60mm] level 2=[sibling distance=5cm] June two Modja [->] child[sibling distance=37mm] node (dp) DePalmas two Higelin three June child[sibling distance=12mm] node (lf) LeForestier two Modja child[sibling distance=30mm] node (mo) Moustaki two Vian three ;

[thick,>=stealth,->] (dp)–+(lf); [thick,>=stealth,->] (lf)–+(mo);

Insertion de Berger

bplus=[rectangle split, rectangle split horizontal,rectangle split ignore empty parts,draw,minimum height=0.6cm] every node=[bplus] level 1=[sibling distance=60mm] level 2=[sibling distance=5cm] June two Modja [->] child[sibling distance=42mm] node (b) Berger two DePalmas three Higelin four June child[sibling distance=12mm] node (lf) LeForestier two Modja child[sibling distance=30mm] node (mo) Moustaki two Vian three ;

[thick,>=stealth,->] (b)–+(lf); [thick,>=stealth,->] (lf)–+(mo);

Insertion de Goldman

bplus=[rectangle split, rectangle split horizontal,rectangle split ignore empty parts,draw,minimum height=0.6cm] every node=[bplus] level 1=[sibling distance=60mm] level 2=[sibling distance=5cm] Goldman two June three Modja [->] child[sibling distance=37mm] node (b) Berger two DePalmas three Goldman child[sibling distance=42mm] node (h) Higelin two June child[sibling distance=13mm] node (lf) LeForestier two Modja child[sibling distance=24mm] node (mo) Moustaki two Vian three ;

[thick,>=stealth,->] (b)–+(h); [thick,>=stealth,->] (h)–+(lf); [thick,>=stealth,->] (lf)–+(mo);

Insertion de Mitchell, Katerine et Estefan

bplus=[rectangle split, rectangle split horizontal,rectangle split ignore empty parts,draw,minimum height=0.6cm] every node=[bplus] level 1=[sibling distance=60mm] level 2=[sibling distance=5cm] Goldman two June three Modja [->] child[sibling distance=42mm] node (b) Berger two DePalmas three Estefan four Goldman child node (h) [right= 0.15 cm of b] Higelin two June child node (k)[right= 0.15 cm of h] Katerine two LeForestier three Mitchell four Modja child node (mo) [right= 0.15 cm of k] Moustaki two Vian three ; [thick,>=stealth,->] (b)–+(h); [thick,>=stealth,->] (h)–+(k); [thick,>=stealth,->] (k)–+(mo);

Insertion de Marley

bplus=[rectangle split, rectangle split horizontal,rectangle split ignore empty parts,draw,minimum height=0.6cm] every node=[bplus] level 1=[sibling distance=60mm] level 2=[sibling distance=5cm] Goldman two June three Marley four Modja [->] child[sibling distance=42mm] node (b) Berger two DePalmas three Estefan four Goldman child node (h) [right= 0.15 cm of b] Higelin two June child node (k)[right= 0.15 cm of h] Katerine two LeForestier three Marley child node (mi)[right= 0.15 cm of k] Mitchell two Modja child node (mo) [right= 0.15 cm of mi] Moustaki two Vian three ;

[thick,>=stealth,->] (b)–+(h); [thick,>=stealth,->] (h)–+(k); [thick,>=stealth,->] (k)–+(mi); [thick,>=stealth,->] (mi)–+(mo);

Insertion de Azrié

bplus=[rectangle split, rectangle split horizontal,rectangle split ignore empty parts,draw] every node=[bplus] level 1=[sibling distance=60mm] level 2=[sibling distance=15mm] June [->] child node (dp) DePalmas two Goldm. child node (a) Azrié two Berger three DePalm. child node (e) [right= 0.15 cm of a] Estefan two Goldm. child node (h) [right= 0.15 cm of e] Higelin two June child node (m) Marley two Modja child node (k) [right= 0.15 cm of h] Katerine two LeFor. three Marley child node (mi) [right= 0.15 cm of k] Mitchell two Modja child node (mo) [right= 0.15 cm of mi] Moustaki two Vian ; [thick,>=stealth,->] (a)–+(e); [thick,>=stealth,->] (e)–+(h); [thick,>=stealth,->] (h)–+(k); [thick,>=stealth,->] (k)–+(mi); [thick,>=stealth,->] (mi)–+(mo);

7.2.2 On considère que cet arbre B+ est stocké à raison d'un nœud par page sur disque. On recherche les noms des artistes dont l'initiale du nom se trouve entre 'B' et 'I' (inclus). Combien de pages disque doivent être chargées au minimum pour répondre à cette requête en utilisant l'arbre B+ ? Justifier. (1 point)

Il s'agit ici d'une requête par intervalle, facilement réalisable avec un arbre B+. On commence par faire une recherche du premier enregistrement dont l'initiale du nom est égale ou immédiatement supérieure à 'B'. On a trouvé le premier enregistrement répondant à la requête (en parcourant ici 3 feuilles de l'arbre B+), il suffit d'exploiter le chaînage des feuilles pour trouver les autres enregistrements (en parcourant ici encore 2 feuilles de l'arbre B+). On charge donc 5 pages d'index. Il faut ensuite récupérer les données pointées par l'index sur disque. Au mieux, toutes les données sont stockées dans une même page (peu probable mais possible...). Donc on charge au minimum 6 pages.

7.3 Optimisation (4 points)

Soit la requête suivante:

```
Select id_abonne, ALBUM.nom
From ECOUTE, CHANSON, ALBUM
Where ECOUTE.id_chanson = CHANSON.id
      AND CHANSON.id_album = ALBUM.id
      AND date=29/05/2013;
```

7.3.1 On suppose qu'il existe des index sur les attributs suivants (et seulement sur ceux-ci) :

- `id_abonne` de ECOUTE,
- `id` de CHANSON, et
- `id_chanson` de ECOUTE,

Donner un plan d'exécution (arborescente ou EXPLAIN) produit par l'optimiseur pour la requête (indiquez les prédicats de sélection sur ce plan). **Argumenter** votre réponse. (2 points) L'index sur ECOUTE(id_abonne) ne sert à rien dans cette requête. Les deux index suivants peuvent être utilisés pour la jointure entre CHANSON et ECOUTE (donc boucles imbriquées avec index) mais pas tous les deux en même temps, il faut choisir l'un des deux (les deux options sont acceptées ici). Ensuite tri-fusion ou boucles-imbriquées acceptés pour la seconde jointure (dépend de la taille des tables). Ci-dessous, un tri-fusion pour la seconde jointure.

```
0 SELECT STATEMENT
1*   MERGE JOIN
2      SORT JOIN
3*        NESTED LOOPS
4*          TABLE ACCESS FULL           ECOUTE
5           TABLE ACCESS BY ROWID       CHANSON
6              INDEX RANGE SCAN         IDX-CHANSON_ID
7      SORT JOIN
8         TABLE ACCESS FULL             ALBUM

1 - access(CHANSON.id_album=ALBUM.id)
3 - access(ECOUTE.id_chanson=CHANSON.id)
4 - access(date=29/05/2013)
```

Attention, on rappelle ici que les choix doivent être argumentés (donner le plan seul ne suffit pas).

7.3.2 On ajoute maintenant (en plus des index de la question précédente) les index suivant :

- un index sur l'attribut `id` de ALBUM et
- un index sur l'attribut `date` de ECOUTE.

Donner un plan d'exécution (arborescente ou EXPLAIN) pour la requête produit par l'optimiseur (privilégiant l'utilisation des index). **Argumenter** votre réponse. (2 points) L'index sur ALBUM(id) est utilisé pour la seconde jointure qui devient donc une jointure par boucles imbriquées avec index. L'index sur ECOUTE(date) n'est pas utilisé pour la jointure entre ECOUTE et CHANSON mais pour effectuer une sélection des enregistrements sur la table directrice ECOUTE dans la première jointure.

```
0 SELECT STATEMENT
1*    NESTED LOOPS
2*        NESTED LOOPS
3            TABLE ACCESS BY ROWID       ECOUTE
4*             INDEX RANGE SCAN          IDX-ECOUTE_DATE
5            TABLE ACCESS BY ROWID       CHANSON
6              INDEX RANGE SCAN          IDX-CHANSON_IDCHANSON
7         TABLE ACCESS BY ROWID          ALBUM
8              INDEX RANGE SCAN          IDX_ALBUM_ID

1 - access(CHANSON.id_album=ALBUM.id)
2 - access(ECOUTE.id_chanson=CHANSON.id)
4 - access(date=29/05/2013)
```

7.4 Concurrence (3 points)

Soit l'histoire suivante :
H = $r_2[x]\ r_3[x]\ w_1[y]\ r_3[y]\ w_3[y]\ r_1[z]\ w_2[y]\ c_1\ w_3[z]\ w_2[z]\ c_3\ c_2$

7.4.1 Donner la liste des conflits de H ; `(0,5 point)`

sur x : aucun conflit

sur y : $w_1[y]r_3[y]$, $w_1[y]w_3[y]$, $r_3[y]w_2[y]$, $w_1[y]w_2[y]$ et $w_3[y]w_2[y]$

sur z : $w_3[z]w_2[z]$

7.4.2 Donner le graphe de sérialisation de H. Que pouvez-vous déduire de ce graphe ? `(0,5 point)` $T_1 \rightarrow T_3, T_3 \rightarrow T_2, T_1 \rightarrow T_2$. Il n'y a pas de cycle, dont H est *sérialisable*.

7.4.3 Donner l'histoire finale par application de l'algorithme de verrouillage à deux phases. Donner le détail du déroulement de l'algorithme. On suppose que les verrous sont relâchés immédiatement après les *commit*. `(2 points)`

I	Verrous			Opérations			En attente	*Commentaires*
	x	y	z	T_1	T_2	T_3		
(1)	rl_2			$r_2[x]$				
(2)	rl_2, rl_3				$r_3[x]$			
(3)	rl_2, rl_3	wl_1		$w_1[y]$				
(4)	rl_2, rl_3	wl_1					$r_3[y]$	T_3 est mise en attente (elle ne peut pas poser de verrou sur y)
(5)	rl_2, rl_3	wl_1					$r_3[y]$ $w_3[y]$	T_3 en attente donc $w_3[y]$ en attente (préservation de l'ordre des opérations au sein d'une transaction)
(6)	rl_2, rl_3	wl_1	rl_1	$r_1[z]$			$r_3[y]$ $w_3[y]$	
(7)	rl_2, rl_3	wl_1	rl_1				$r_3[y]$ $w_3[y]$ $w_2[y]$	T_2 mise en attente
(8)	rl_2, rl_3			c_1			$r_3[y]$ $w_3[y]$ $w_2[y]$	T_1 libère tous ses verrous
(9)	rl_2, rl_3	rl_3				$r_3[y]$	$w_3[y]$ $w_2[y]$	Traitement prioritaires des opérations en attente, T_3 est débloquée
(10)	rl_2, rl_3	wl_3				$w_3[y]$	$w_2[y]$	"Montée" de verrou partagé en verrou exclusif par T_3 sur y
(11)	rl_2, rl_3	wl_3	wl_3			$w_3[z]$	$w_2[y]$	T_2 toujours bloquée
(12)	rl_2, rl_3	wl_3	wl_3			$w_3[z]$	$w_2[y]$ $w_2[z]$	T_2 toujours bloquée donc mise en attente de toutes ses opérations
(13)	rl_2					c_3	$w_2[y]$ $w_2[z]$	T_3 relâche tous ses verrous
(14)	rl_2	wl_2			$w_2[y]$		$w_2[z]$	T_2 débloquée
(15)	rl_2	wl_2	wl_2		$w_2[z]$			
(16)			c_2					

rl: read lock, verrou partagé (pour lecture).
wl : write lock, verrou exclusif (pour écriture).

Ce qui nous donne l'exécution suivante :
H' = $r_2[x]$ $r_3[x]$ $w_1[y]$ $r_1[z]$ c_1 $r_3[y]$ $w_3[y]$ $w_3[z]$ c_3 $w_2[y]$ $w_2[z]$ c_2

8 | NFP 107 HTT, Septembre 2013

8.1 Algèbre relationnelle et SQL (10 points)

Soit l'extrait suivant de la base de données qui sert à la gestion de la bibliothèque de la ville de Metropolis :

```
   LIVRE(id, titre, id_auteur, date_publication, catégorie, éditeur)
EMPRUNT(id_personne, id_livre, date_début, date_fin)
PERSONNE(id, nom, prénom, age)
AUTEUR(id, nom, prénom, date_naissance, date_décés, nationalité)
```

8.1.1 Pour la table LIVRE, identifiez les clés primaires et étrangères. (0,5 points) Clé primaire: id
Clés étrangères: id_auteur → AUTEUR(id)

8.1.2 Donnez la commande SQL pour créer la table LIVRE. (0,5 points)

```
CREATE TABLE LIVRE(
    id NUMBER(10),
    titre VARCHAR2(50),
    id_auteur NUMBER(10),
    date_publication DATE,
    catégorie VARCHAR2(20),
    éditeur VARCHAR2(20),
    CONSTRAINT pk_livre PRIMARY_KEY id,
    CONSTRAINT fk_livre FOREIGN_KEY id_auteur REFERENCES AUTEUR(id)
```

8.1.3 (Algèbre et SQL) Donnez les titres de livres publiés avant 01/01/2010 chez l'éditeur "Henry Lemoine". (1 point)

$\pi_{titre}(\sigma_{editeur='Henry\ Lemoine'\ AND\ date<='01/01/2010'}(LIVRE))$

```
SELECT titre FROM LIVRE
WHERE editeur = 'Henry Lemoine' AND date <= '01/01/2010'
```

8.1.4 (Algèbre et SQL) Donnez les noms des personnes qui ont emprunté des livres de la catégorie 'thriller' écrits par un auteur français. (3 point)

$A = \rho_{id/id_auteur}(AUTEUR)$
$L = \rho_{id/id_livre}(LIVRE)$
$P = \rho_{id/id_personne}(PERSONNE)$
$\pi_{titre}(\sigma_{nationalite='francaise'\ AND\ categorie='thriller'}(A \bowtie (P \bowtie (L \bowtie EMPRUNT))))$

```
SELECT nom FROM EMPRUNT e, LIVRE l, AUTEUR a, PERSONNE p
WHERE  AND e.id_personne = p.id AND e.id_livre = l.id
       AND l.id_auteur = a.id AND l.categorie = 'thriller'
       AND a.nationalite = 'francaise';
```

8.1.5 (SQL seulement) Donnez la moyenne d'âge des personnes qui ont emprunté des livres de la catégorie 'jeunesse' parus en 2013 (2 points):

```
SELECT AVG(age) FROM EMPRUNT e, LIBRE l, PERSONNE p
WHERE e.id_livre = l.id AND e.id_personne = p.id
      AND l.categorie = 'jeunesse' AND date_publication >= '01/01/2013'
      AND date_publication <= '31/12/2013'
```

8.1.6 Donnez les noms des personnes qui ont emprunté le plus de livres (3 points).

```
SELECT p.nom FROM EMPRUNT e, PERSONNE p
WHERE e.id_personne = p.id
GROUP BY id_personne
HAVING COUNT(*) = (SELECT MAX(COUNT(*))
                   FROM EMPRUNT
                   GROUP BY id_personne)
```

8.2 Indexation (3 points)

On insère successivement les enregistrements suivants dans la table `PERSONNE`, selon cet ordre :

id	nom	prénom	**age**
1	Martin	Georges	30
2	Dubois	Inna	20
3	Thomas	Maxime	14
4	Richard	Boris	45
5	Petit	Gérald	32
6	Durand	Valérie	43
7	Leroy	Jacques	34
8	Simon	Michel	22
9	Laurent	Jean-Jacques	63
10	Moreau	Eddie	12
11	Morel	Philippe	15
12	Perrin	Gloria	18
13	Clement	Bob	19
14	Dumont	Claire	67

On met en place un index sur l'attribut **âge** de cette table.

8.2.1 Construire l'arbre B+ d'ordre **2** correspondant à cet ensemble d'articles, en respectant l'ordre d'arrivée. Donner les étapes de construction intermédiaires importantes (celles qui produisent un changement de la structure de l'arbre). (2 points)

L'arbre final est le suivant:

bplus=[rectangle split, rectangle split horizontal,rectangle split ignore empty parts,draw,minimum height=0.4cm] every node=[bplus] level 1=[sibling distance=60mm] level 2=[sibling distance=5cm] 15 two 20 three 30 four 43 [->] child[sibling distance=25mm] node (b) 12 two 14 three 15 child node (h) [right= 1 cm of b] 18 two 19 three 20 child node (k)[right= 1 cm of h] 20 two 30 child node (mi)[right= 1 cm of k] 32 two 34 three 43 child node (mo) [right= 1 cm of mi] 45 two 63 three 67 ;

[thick,>=stealth,->] (b)–+(h); [thick,>=stealth,->] (h)–+(k); [thick,>=stealth,->] (k)–+(mi); [thick,>=stealth,->] (mi)–+(mo);

NB pour la suite: les feuilles de l'arbre B+ "pointent" vers le fichier de données n'apparaissant pas ici.

8.2.2 On considère que cet arbre B+ est stocké à raison d'un noeud par page sur disque. On recherche les noms des personnes dont l'âge est entre 20 et 40 ans (inclus). Combien de pages disque doivent être chargées au minimum pour répondre à cette requête en utilisant l'arbre B+ ? Justifier. (1 point)

Il s'agit ici d'une requête par intervalle: on commence par chercher les enregistrements pour lesquels l'âge est égal à 20 (lecture de 5 enregistrements). Il suffit ensuite d'exploiter le chaînage des feuilles pour trouver les autres enregistrements, en parcourant ici encore 5 enregistrements dans l'arbre B+). On charge donc 10 pages d'index. Il faut ensuite récupérer les données pointées par l'index sur disque. Dans le meilleur cas (peu probable mais possible), toutes ces données sont stockées dans une même page. Donc on charge au minimum 11 pages.

8.3 Optimisation (4 points)

Soit la requête suivante:

```
Select PERSONNE.id, PERSONNE.nom, PERSONNE.prénom
From LIVRE, EMPRUNT, PERSONNE, AUTEUR
Where AUTEUR.nom = "Cioran"
      AND LIVRE.id_auteur = AUTEUR.id
      AND EMPRUNT.id_livre = LIVRE.id
      AND PERSONNE.id = EMPRUNT.id_personne;
```

8.3.1 On suppose qu'il existe des index sur les attributs suivants (et seulement sur ceux-ci) :

- `nom` de AUTEUR,
- `id` de LIVRE, et
- `id_livre` de EMPRUNT,

Donner un plan d'exécution (arborescente ou EXPLAIN) produit par l'optimiseur pour la requête (indiquez les prédicats de sélection sur ce plan). **Argumenter** la réponse. (2 points)

8.3.2 On ajoute maintenant (en plus des index de la question précédente) l'index suivant:

- un index sur l'attribut `id_personne` de EMPRUNT.

Donner un plan d'exécution (arborescente ou EXPLAIN) pour la requête produit par l'optimiseur (privilégiant l'utilisation des index). **Argumenter** la réponse. (2 points)

8.4 Concurrence (3 points)

Soit l'histoire suivante :
H = $r_1[x]r_2[x]r_3[y]w_1[x]w_1[z]c_1w_2[y]c_2w_3[z]c_3$

8.4.1 Donner la liste des conflits de H; `(0,5 point)`

sur x : $r_2[x]w_1[x]$

sur y : $r_3[y]w_2[y]$

sur z : $w_1[z]w_3[z]$

8.4.2 Donner le graphe de sérialisation de H. Que pouvez-vous déduire de ce graphe? `(0,5 point)` $T_2 \rightarrow T_1, T_3 \rightarrow T_2, T_1 \rightarrow T_3$. Il y a un cycle, dont H n'est pas *sérialisable*.

8.4.3 Donner l'histoire finale par application de l'algorithme de verrouillage à deux phases. Donner le détail du déroulement de l'algorithme. On suppose que les verrous sont relâchés immédiatement après les *commit*. `(2 points)`

9 | NFP 107 FOD, 12 février 2014

Voici une partie de la base de données utilisée pour gérer un site de ventes en ligne. La société dispose de plusieurs entrepôts, un par département (de France).

`PRODUIT`(id, nom, prix, catégorie, fournisseur, marque)
`CLIENT`(id, nom, prénom, adresse, département)
`COMMANDE`(id_client, id_produit, date, quantité)
`STOCK`(id_produit, département, stock)

9.1 Algèbre relationnelle et SQL (10 points)

9.1.1 Identifiez les clés primaires et étrangères pour la table STOCK. (0.5 points)
Correction :

Cles primaire : (id_produit, departement)
Cles etrangeres : id_produit−>PRODUIT(id)

9.1.2 Donnez la commande SQL pour créer la table STOCK. (0.5 points)
Correction :

```
CREATE TABLE STOCK(
        id_produit INTEGER,
        departements VARCHAR(20),
        quantite_en_stock,
        CONSTRAINT pk_stock PRIMARY_KEY (id_produit, departement),
        CONSTRAINT fk_stock FOREIGN_KEY (id_produit)
                REFERENCES PRODUIT(id))
```

9.1.3 Donnez les noms et prénoms des clients qui achètent des produits de la marque 'Apple' (Algèbre et SQL, 1 point)
Correction :

```
SELECT nom, prenom FROM COMMANDE com, CLIENT cl, PRODUIT p
WHERE com.id_client = cl.id AND
        com.id_produit = p.id AND
        p.marque = 'Apple';
```

9.1.4 Donner les noms et prénoms des clients dont une commande ne peut être satisfaites (la quantité demandée dépasse le stock disponible dans leur département). (Algèbre et SQL, 2 points)
Correction :

```
SELECT nom, prenom
FROM CLIENT cl, COMMANDE com
WHERE cl.id = com.id_client AND
        quantite >
                (SELECT quantite_en_stock
                FROM STOCK s WHERE s.id_produit = com.id_produit
                AND s.departement = cl.departement)
```

9.1.5 Donnez les noms des produits qui ne sont pas disponibles dans aucun entrepôt (SQL seulement, 2 points)
Correction :

```
SELECT p.nom FROM PRODUIT p, COMMANDE c
WHERE c.id_produit = p.id AND
        c.id_produit NOT IN (SELECT DISTINCT id_produit FROM STOCK);
```

9.1.6 Donner les nom et prénoms des clients ayant commandé plus de 10 agendas aujourd'hui. (SQL seulement, 2 points)
Correction :

```
SELECT nom, prenom FROM COMMANDE com, CLIENT cl, PRODUIT p
WHERE com.id_client = cl.id AND
        com.id_produit = p.id AND p.nom = 'Agenda'
GROUP BY id_client
HAVING SUM(quantite) > 10ï¿½;
```

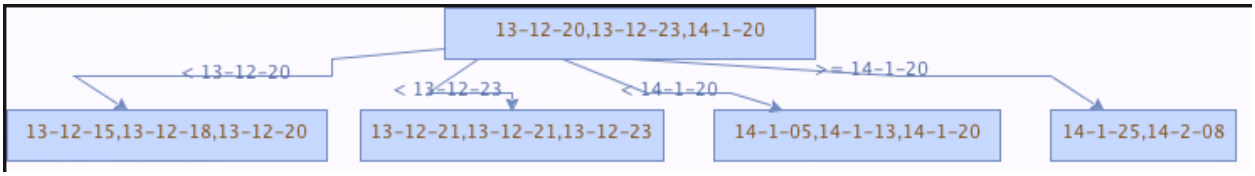
9.1.7 Donnez les noms et prénoms des clients qui ont acheté un des produits les plus chers de la marque 'Samsung'. Attention ! : plusieurs produits peuvent avoir le même prix. (SQL seulement, 2 points)
Correction :

```
SELECT DISTINCT nom, prenom
FROM COMMANDE com, CLIENT cl, PRODUIT p
WHERE marque = 'Samsung' AND
        id_produit = p.id AND id_client = cl.id AND
        p.id_produit IN
```

(**SELECT MAX**(prix) **FROM** PRODUIT
WHERE marque = 'Samsung')

9.2 Indexation (3 points)

9.2.1 On souhaite crÃ©er une index B+Tree d'ordre 2 sur l'attribut `COMMANDE.date`. Construire l'arbre correspondant Ã l'ensemble de valeurs ci-dessous, en respectant l'ordre d'arrivÃ©e. Donner les Ã©tapes de construction intermÃ©diaires importantes (celles qui produisent un changement de la structure de l'arbre). (2 points)
`2013-12-15, 2013-12-23, 2013-12-20, 2014-01-13, 2013-12-18, 2013-12-21, 2014-01-20, 2013-12-21, 2014-01-05, 2014-02-08, 2014-01-25`
Correction :



9.2.2 On souhaite rï¿½cupï¿½rer toutes les commandes passï¿½es depuis le '01/01/2014'. Si un noeud de l'index est une page sur le disque, combien de pages de l'index doivent ï¿½tre lues pour rï¿½pondre ï¿½ cette requï¿½te ? (1 point)
Correction :
Il faut chercher la plus grande valeur avant le '01/01/2014' et parcourir les pages ï¿½ partir du noeud trouvï¿½ jusqu'ï¿½ la derniï¿½re feuille. Ainsi, on va trouver le noeud contenant '2013-12-23', puis parcourir les deux autres feuilles. Ce qui nous fait au total 4 pages d'index (1xracine, 1xrecherche, 2xparcours).

9.3 Optimisation (4 points)

9.3.1 (1 point) Pour la requï¿½te ci-dessous, dire ce quelle produit comme rï¿½sultat.

SELECT nom
FROM Produit P, Commande C
WHERE P.id=C.id_produit
    AND prix > 100
    AND quantite > 10
    AND marque = 'Bricoflex';

Correction :
Donner les noms des produits de marque Bricoflex ayant un prix supï¿½rieur ï¿½ 100 (l'unitï¿½), ayant une commande de plus de 10 unitï¿½s (chaque commande).

9.3.2 (3 points) Donner un plan d'exï¿½cution (EXPLAIN et arborescent) sachant qu'il existe un index sur l'attribut `marque` de la table `Produit`, qu'il n'y a pas d'autres index et que les deux tables sont de tailles moyennes. Ne pas oublier d'ajouter l'emplacement des diffï¿½rents filtres de la requï¿½te.
Correction :

```
0  SELECT STATEMENT
1*   NESTED LOOPS
2*     TABLE ACCESS BY INDEX ROWID   Produit
3*       INDEX RANGE SCAN            IDX_Marque
4*     TABLE ACCESS FULL             Commande

(1) P.id=C.id_produit
(2) prix > 100
(3) marque = 'Bricoflex'
(4) quantite > 10
```

9.4 Concurrence (3 points)

Soit l'exécution concurrente suivante de trois transactions dans un SGBD.
H: $r_1[x]r_2[x]r_1[y]w_2[y]r_3[x]w_1[x]r_3[y]c_1c_2a_3$

9.4.1 Existe-t-il une "lecture sale" ? Pourquoi faut-il éviter ce type de lecture (expliquer les effets possibles) ? (1 point)

9.4.2 Quelle est l'exécution obtenue par verrouillage à deux phases à partir de H? On considère que le relachement des verrous d'une transaction se fait au Commit et qu'à ce moment on exécute en priorité les opérations bloquées en attente de verrou, dans l'ordre de leur blocage. (2 points)

10 | NFP 107 FOD, 9 avril 2014

10.1 Algèbre relationnelle et SQL (10 points)

Soit l'extrait suivant de la base de données qui sert à gérer un parc d'avions :

```
   VOYAGE(numVol, villeDepart, villeArrivée, dateDepart, dateArrivée)
VOL(numVol, compagnie, numAvion, nombreVoyageurs)
AVION(numAvion, modèle, capacité, consommation, reserveCombustible)
DISTANCE(ville1, ville2, dist)
VILLE(nomVille, population, continent)
```

10.1.1 Pour la table DISTANCE, identifiez les clés primaires et étrangères. (0,5 points)
Correction :
Clé primaire: (ville1, ville2)
Clés étrangères: ville1,ville2 → VILLE(nomVille)

10.1.2 Donnez la commande SQL pour créer la table DISTANCE. (0,5 points)
Correction :

```
CREATE TABLE DISTANCE(
    ville1 VARCHAR2(10),
    ville2 VARCHAR2(10),
    population NUMBER(10),
    continent VARCHAR2(10),
    CONSTRAINT pk_distance PRIMARY_KEY (ville1, ville2)
    CONSTRAINT fk1_distance FOREIGN_KEY ville1 REFERENCES VILLE(nomVill
    CONSTRAINT fk2_distance FOREIGN_KEY ville2 REFERENCES VILLE(nomVill
```

10.1.3 (Algèbre et SQL) Donnez les vols qui durent plus de 10 heures.(1 point)
Correction :

$\sigma_{dateArrive-dateDepart>10\ heures}(VOYAGE)$

```
SELECT * FROM VOYAGE
WHERE (dateArrivée - dateDepart)*24 > 10;
```

10.1.4 (Algèbre et SQL) Les voyages qui embarquent plus de 400 voyageurs. (1 point)
Correction :

$\sigma_{nombreVoyageurs>400}(VOYAGE \bowtie VOL)$

```
SELECT * FROM VOYAGE voy, VOL v
WHERE (voy.numVol=v.numVol AND nombreVoyageurs > 400;
```

10.1.5 (Algèbre et SQL seulement) Nombre de vols au départ de Paris vers l'Afrique qui embarquent moins de 50% de leur capacité (3 points):
Correction :

$$\sigma_{villeDepart='Paris' \wedge continent='Afrique' \wedge nbVoyageurs<capacite/2} \\ (((VOYAGE \bowtie VOL) \bowtie AVION) \bowtie_{villeArrivee=nomVille} VILLE) \quad (10.1)$$

```
SELECT COUNT(voy.numVol)
FROM VOYAGE voy, VOL v, AVION a, VILLE vl
WHERE voy.numVol = v.numVol AND v.numAvion = a.numAvion AND
      villeDepart = 'Paris' AND villeArrivee = vl.nomVille AND
      v.continent = 'Afrique' AND nbVoyageurs < capacite/2;
```

10.1.6 (SQL seulement) Le modèle d'avion qui transporte le plus de voyageurs (tous vols compris) (2 points).

Correction :

```
SELECT modele
FROM VOL v, AVION a
WHERE v.numAvion = a.numAvion
GROUP BY modele
HAVING SUM(nombreVoyageurs) = (SELECT MAX(SUM(nombreVoyageurs))
                               FROM VOL v, AVION a
                               WHERE v.numAvion = a.numAvion
                               GROUP BY modele)
```

10.1.7 (Algèbre seulement) Quelle est la (ou les) ville(s) destination vers laquelle toutes les compagnies ont des vols ? (1 point).

Correction :
$\pi_{villeArrivee,compagnie}(VOYAGE \bowtie VOL) \div \pi_{compagnie}(VOL)$

10.1.8 (Algèbre seulement) Quels sont les modèles d'avion qui consomment moins que tous les avions de la compagnie AIR FRANCE? (1 point).

Correction :

$$\pi_{modele}(AVION) - \\ \pi_{modele}(AVION \bowtie_{consommation > consommation'} \rho_{consommation/consommation'}(\\ \pi_{consommation}(\sigma_{compagnie='AIRFRANCE'}(VOL) \bowtie AVION))) \quad (10.2)$$

10.2 Indexation (3 points)

10.2.1 Un index est créé sur les identifiants de Vol. Construire l'index d'ordre 2. Montrer la construction de l'index à chaque étape d'éclatement avec les valeurs suivantes : (3 points)
5, 28, 32, 36, 61, 73, 81, 92, 3, 7, 38, 30, 29, 50, 33, 34, 55, 75, 27, 31, 77, 98

10.3 Optimisation (4 points)

Soit la requête suivante :

```
SELECT DISTINCT villeArrivée
FROM Vol V, Voyage Vo
WHERE V.Compagnie = 'Air France' AND
    V.NumVol = Vo.NumVol AND
    Vo.VilleDépart = 'Paris';
```

10.3.1 Donner en une phrase ce que produit cette requête (`1 point`).
Correction :
Cette requête donne les destinations distinctes des vols au départ de Paris pour la compagnie 'Air France'.

10.3.2 Donner <u>deux</u> plans d'exécution EXPLAIN différents à partir de cette requête. Sachant que : (`3 points`)

- l'opération pour le distinct est un *'SORT UNIQUE'*, juste après le *'SELECT STATEMENT'*
- les index existants sont :
  - BTree V.Compagnie
  - BTree Vo.VilleDépart
  - BTree Vo.NumVol

NB: les deux plans doivent proposer deux algorithmes de jointures différents

10.4 Concurrence (3 points)

Soit l'histoire suivante :
H = $r_1[x]r_2[x]r_3[y]w_1[x]w_1[z]c_1w_2[y]c_2w_3[z]c_3$

10.4.1 Donner la liste des conflits de H; `(0,5 point)`
Correction :

sur x : $r_2[x]w_1[x]$

sur y : $r_3[y]w_2[y]$

sur z : $w_1[z]w_3[z]$

10.4.2 Donner le graphe de sérialisation de H. Que pouvez-vous déduire de ce graphe? `(0,5 point)`
Correction :
$T_2 \rightarrow T_1, T_3 \rightarrow T_2, T_1 \rightarrow T_3$. Il y a un cycle, dont H n'est pas *sérialisable*.

10.4.3 Donner l'histoire finale par application de l'algorithme de verrouillage à deux phases. Donner le détail du déroulement de l'algorithme. On suppose que les verrous sont relâchés immédiatement après les *commit*. `(2 points)`

11 | NFP 107 HTO, 18 juin 2014

11.1 Algèbre relationnelle et SQL (10 points)

Soit l'extrait suivant du schéma de la base de données permettant de gérer une société de location de voitures:

```
VEHICULE(idVehicule, modele, gamme, dateAcquisition, kilometrage,
agence)
AGENCE(idAgence, ville, dateOuverture, gerant)
CLIENT(idClient, nom, prenom, datePremiereLocation, dateDerniereLocatio
LOCATION(idVehicule,idClient,gammeDemandee,dateDebut,dateFin,agenceFin)
```

Chaque voiture est affectée à une agence (attribut agence dans VEHICULE). Lors d'une location, un client indique la gamme de voiture (citadine, compacte, familiale, monospace, break, etc.) souhaitée (l'information se trouve dans gammeDemandee), l'agence lui met à disposition un véhicule disponible de cette gamme, sinon d'une gamme supérieure (l'information se trouvera dans l'attribut gamme du véhicule loué). La voiture peut être retournée dans l'agence de départ ou dans une autre agence (agenceFin).

(a) Pour la table LOCATION, identifiez les clés primaires et étrangères. (0,5 points)
Correction :
Si on considère qu'un même client peut louer plusieurs fois le même véhicule, alors la clé primaire pourrait être (idVehicule, idClient, dateDebut).
Les clés étrangères seront: idVehicule → VEHICULE(idVehicule), idClient → CLIENT(idClient), agenceFin → AGENCE(idAgence)

(b) Donnez la commande SQL pour créer la table LOCATION. (0,5 points)
Correction :

```
CREATE TABLE LOCATION(
    idVehicule INTEGER,
    idClient   INTEGER,
    gammeDemandee VARCHAR2(20),
    dateDebut  DATE,
    dateFin    DATE,
    agenceFin  INTEGER,
    CONSTRAINT pk_loc  PRIMARY_KEY (idVehicule, idClient, dateDebut)
    CONSTRAINT fk1_loc FOREIGN_KEY idVehicule REFERENCES VEHICULE(id
    CONSTRAINT fk2_loc FOREIGN_KEY idClient   REFERENCES CLIENT(idCl
    CONSTRAINT fk3_loc FOREIGN_KEY agenceFin  REFERENCES AGENCE(idAg
```

(c) (Algèbre et SQL) Quels véhicules ont un kilométrage supérieur à 10.000 km? (1 point)
Correction :
$\sigma_{kilometrage>10000\ km}(VEHICULE)$

```
SELECT * FROM VEHICULE
WHERE kilometrage > 10000;
```

(d) (SQL seulement) De combien de véhicules de chaque gamme dispose l'agence de Tarbes (dont l'identifiant est 18)? (1 point)
Correction :

```
SELECT gamme, COUNT(idVehicule)
FROM   VEHICULE
WHERE  agence = 18
GROUP  BY gamme;
```

(e) (SQL seulement) Trier les agences par nombre décroissant de surclassements accordés (surclassement: pour une location, la gamme du véhicule loué est différente de gammeDemandee)? (2 points)
Correction :

```
SELECT v.agence, COUNT(*) AS nbSurclassements
FROM LOCATION l, VEHICULE v
WHERE l.idVehicule     = v.idVehicule AND
      l.gammeDemandee != v.gamme
GROUP BY v.agence
ORDER BY nbSurclassements;
```

(f) (Algèbre et SQL) Noms et prénoms des clients ayant loué au moins un véhicule dans l'agence de Nice après le 23/12/2013. (3 points)
Correction :

$$\pi_{nom,prenom}\{CLIENT \bowtie [(\sigma_{dateDebut>23/12/2013}LOCATION) \bowtie (\sigma_{agence=3}VEHICULE)]\} \quad (11.1)$$

```
SELECT c.nom, c.prenom
FROM CLIENT c, LOCATION l, VEHICULE v
WHERE l.dateDebut > 23/12/2013 AND
      v.agence = 3 AND
      l.idVehicule = v.idVehicule AND
      l.idClient = l.idClient;
```

(g) (Algèbre seulement) Quels clients ont loué au moins une voiture de chaque gamme? (2 points)
Correction :
$R1 := \pi_{idClient,gamme}(VEHICULE \bowtie LOCATION)$; signification: gammes des

voitures louées par chaque client; la jointure est naturelle et se fait sur le seul attribut commun, idVehicule.
$R2 := \pi_{gamme} VEHICULE$; signification: ensemble des gammes de voitures de la table VEHICULE.
Résultat $R := R1 \div R2$; signification: clients ayant loué au moins une voiture de chaque gamme.

Pour information, une des requêtes SQL possibles avec l'interprétation suivante: quels sont les clients tels qu'il n'y a aucune gamme de véhicules dans laquelle ils n'ont pas loué de véhicule?

select Client.idClient
from Client
where not exists
  (**select** Vehicule.gamme
   from Vehicule
   where not exists
    (**select** $*$
     from Location
     where Location.idVehicule = Vehicule.idVehicule
      and Location.idClient = Client.idClient));

et une autre possibilité (donne-t-elle toujours les mêmes résultats que la première?):

```
SELECT l.idClient, COUNT(DISTINCT(v.gamme))
FROM   LOCATION l, VEHICULE v
WHERE  l.idVehicule = v.idVehicule
GROUP BY l.idClient
HAVING COUNT(DISTINCT(v.gamme)) = (SELECT COUNT(DISTINCT(gamme))
                                   FROM VEHICULE);
```

11.2 Indexation (3 points)

On crée un index la table `Location` avec la commande suivante :

create unique index on Location (idVehicule, idClient, dateDebut)

Pour chacune des requêtes ci-dessous, indiquez si l'index peut être utilisé (et justifiez votre réponse à chaque fois).

(a) **select** $*$ **from** Location
where idVehicule=398 **and** dateDebut='18−06−2014'

Correction :
Oui, car `idVehicule` est un préfixe de la clé indexée.

(b) **select** $*$ **from** Location
where idClient=3987 **and** dateDebut='18−06−2014'

Correction :
Non, car ni `idClient`, ni `dateDebut` ne sont préfixe de la clé indexée.

(c) **select** nom
from Client **as** c, Location **as** l
where c.idClient = l.idClient
and dateDebut='18−06−2014'

Correction :
Non, car `idClient` n'est pas préfixe de la clé indexée.

11.3 Optimisation (4 points)

(a) (2 pts) Pour chacune des requêtes de la section précédente, indiquez le plan d'exécution que vous attendriez d'un SGBD relationnel.

Vous pouvez représenter un plan d'exécution sous la forme qui vous convient : **explain** (Oracle) ou arbre d'itérateurs.

Correction :

i. Parcours d'index, puis accès direct à la table.
ii. Parcours séquentiel, sauf si l'on crée un index supplémentaire sur `idCLient`
iii. Parcours séquentiel de `Location`, avec sélection sur la date; pour chaque location, accès par l'index au client.

(b) (1 pt) La figure 11.1 montre le plan d'exécution donné par le SGBD pour la requête de jointure. Commentez ce plan et indiquez l'action éventuelle à entreprendre.

Correction :
Tri fusion, donc il manque un index sur `Client.idClient`. Il faut impérativement en créer un.

(c) (1 pt) Pour la requête de jointure, quel(s) index peut-on créer pour obtenir un plan d'exécution qui n'effectue *que* des accès par index, et aucun accès par balayage séquentiel ?

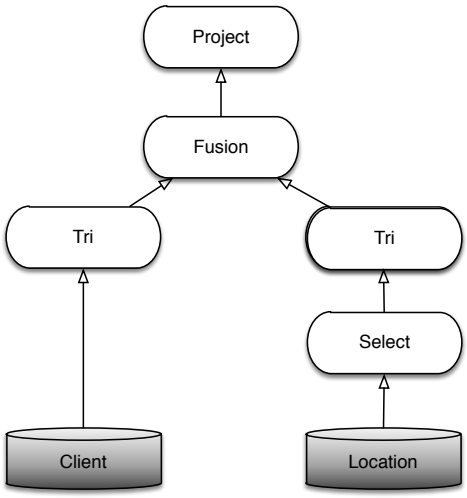


Figure 11.1: Plan d'exécution pour la requête de jointure

Correction :
Il faut impérativement en créer sur `Client.idClient`, et on peut en ajouter un sur `dateDebut`.

11.4 Concurrence (4 points)

Une agence a propose une procédure de location de limousines qui s'effectue pour un véhicule v, un client c, à une date d et pour un nombre n de jours. Elle exécute, dans cet ordre, les requêtes suivantes :

select $*$ **from** Client **where** idClient = $c

insert into Location **values** ($v, $c, 'Limousine', $d, $d+$n, $a)

update Client **set** dateDernierelocation=$d **where** idClient = $c

(a) (0,5 pts) L'exécution concurrente ci-dessous peut-elle représenter l'entrelacement de deux exécutions de la procédure de location ?

$$H_1 = r_1[c_1]r_2[c_2]w_1[l_1]w_1[c_2]w_2[l_2]w_2[c_2]$$

Correction :
Non, car T_1 modifie un client c_2 après avoir lu un client c_1. Ce n'est pas possible avec la procédure de location.

(b) (0,5 pts) Même question pour l'exécution suivante.

$$H_2 = r_1[c_1]r_2[c_1]w_1[l_1]w_1[c_1]C_1w_2[l_2]w_2[c_1]C_2$$

(c) (0,5 pts) Donner la liste des conflits de H_2.

Correction :

sur x : $r_1[c_1]w_2[c_1]$
sur y : $r_2[c_1]w_1[c_1]$
sur z : $w_1[c_1]w_2[c_1]$

(d) (0,5 pts) Donner le graphe de sérialisation de H_2. Que pouvez-vous déduire de ce graphe?
Correction :
$T_2 \rightarrow T_1, T_2 \rightarrow T_1$. Il y a un cycle, dont H n'est pas *sérialisable*.

(e) (1 pt) Donner l'exécution obtenue par application de l'algorithme de verrouillage à deux phases. Donner le détail du déroulement de l'algorithme. On suppose que les verrous sont relâchés immédiatement après les *commit*.

Correction :

- Blocage de T_1 sur $w_1[c_1]$
- Blocage de T_2 sur $w_2[c_1]$
- *Deadlock*!

(f) (1 pt) Que se passe-t-il si on écrit la première requête de la manière suivante ?

select $*$ **from** Client **where** idClient = $c for **update**

Correction :
Verrou exclusif sur la lecture: plus d'interblocage.

12 | NFP 107 HTT, 10 Septembre 2014

12.1 Algèbre relationnelle et SQL (10 points)

Soit la base de données suivante servant pour un site de publication de photos en ligne (type Flickr) :

```
PHOTO(idPhoto, nomPropriétaire, appareilPrise, résolution, lieuPrise,
légende)
PUBLICATION(idPhoto, datePublication, statut)
PROPRIETAIRE(nom, pays, dateInscription, nombrePhotosPubliées)
```

Le nom des attributs est explicite. Le statut d'une photo désigne le fait qu'elle soit visible par tous les utilisateurs (statut = public) ou par les utilisateurs autorisés par son propriétaire (statut = privé).

(a) (1 point)

12.2 Index et Optimisation (5 points)

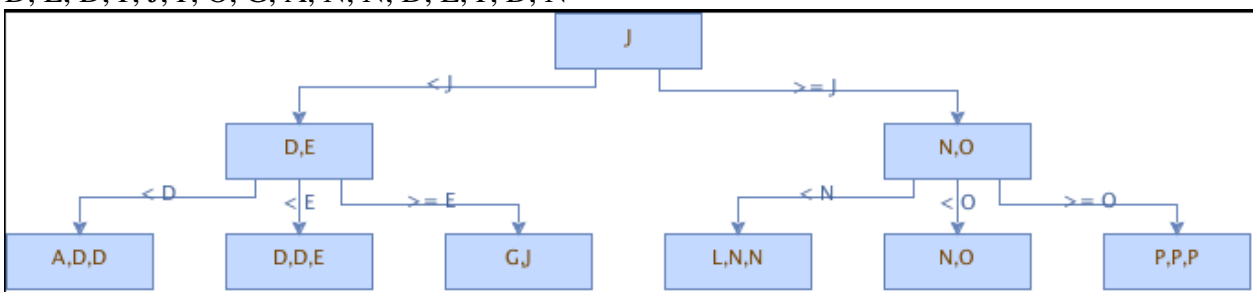
(a) (2.5 points) Créer un index B+Tree d'ordre 2 sur les nom de propriétaire de photos (PHOTO. nomProprietaire). Toute valeur en doublon apparaîtra plusieurs fois. Vous montrerez chaque étape d'éclatement de l'arbre.

Dupont, Lopez, Dupont, Poquelin, Jaquemin, Poquelin, Olivera, Guillard, Achard, Nguyen, Nguyen, Dupont, Echasse, Poquelin, Dupont, Achard, Nguyen

Pour plus de simpliciter, vous pourrez remplacer chaque nom par la première lettre de celui-ci.

Correction :

D, L, D, P, J, P, O, G, A, N, N, D, E, P, D, N



(b) (2.5 points) Pour la requête ci-dessous, dire ce quelle produit comme résultat et donner un plan d'exécution (EXPLAIN et arborescent) en utilisant l'index créé précédemment.

SELECT lieuPrise, légende
FROM Photo Ph, Proprietaire Pr
WHERE Ph.nomProprietaire = Pr.nom
AND Pays='France'
AND RESOLUTION='4272 x 2848';

Correction :
Cette requête donne les lieux de prise et les légendes des photos avec une résolution de '4272 x 2848' prises par des Français

```
0  SELECT STATEMENT
1    NESTED LOOPS
2*     TABLE ACCESS FULL              Proprietaire
3*     TABLE ACCESS BY INDEX ROWID    Photo
4*       INDEX RANGE SCAN             Photo_nomProprietaire

(2) Pays = 'France'
(3) RESOLUTION='4272 x 2848'
(4) Ph.nomProprietaire = Pr.nom
```

12.3 Concurrence (5 points)

Voici une procédure de chargement d'une photo pour un propriétaire nommé n.

```
Procédure charger(Photo photo, String n, Statut s)
début
  // Trouvons la date du jour
  d := dateDuJour()
  // Cherchons le propriétaire
  p := select * from Propriétaire where nom = :n
  // Si on l'a trouvé: mise à jour
  if (p != null) alors
    update Propriétaire set nombrePhotosPubliées = :p.nombrePhotosPubliées + 1
    insert into Photo values (:photo)
    commit
  fait

   insert intoPublication values (:photo.id, :d, :s)
   commit
fin
```

(a) (1,5 point) Critiquez cette procédure, et notamment l'emplacement des `commit`. Signalez deux cas d'anomalie et proposez une correction de la procédure.

(b) (1 point) On exécute deux fois la procédure avec `charger (p1, n, 'public')` et `charger (p2, n, 'public')`. On obtient l'exécution concurrente suivante (u_1 et u_2 désignent les lignes de `Publication` créées pendant l'exécution):

$$H = r_1(n)r_2(n)w_1(n)w_1(p_1)w_2(n)w_2(p_2)w_2(u_2)w_1(u_1)C_1C_2$$

Citer un exemple d'anomalie entrainée par cette exécution.

(c) (1 pt) Montrer que l'exécution précédente n'est pas sérialisable.

(d) (1,5 pt) Appliquer l'algorithme de verrouillage à deux phases à H.

Correction :

(a) Le premier `commit` est effectué alors que la transaction n'est pas complète.

- Si plantage après le premier `commit`: la photo est insérée mais pas la date de publication.
- Si pas de propriétaire: on insère quand même une date de publication et on valide.

Il faut ajouter le second `insert` dans le corps du test, avant le premier `commit`, et supprimer le second `commit`.

(b) Clairement la mise à jour de la seconde transaction écrase la première: le nombre de photos augmente de 1 pour 2 photos insérées.

(c) Conflits $r_2(n) \rightarrow w_1(n)$ et $w_1(n) \rightarrow w_2(n)$, donc non sérialisable.

(d) Cas typique où on aboutit à un interblocage.

13 | NFP 107 HTT, 24 juin 2015

13.1 Algèbre relationnelle et SQL (10 points)

Soit la base de données suivante d'un service de véhicules avec chauffeur : CHAUFFEURS(id, nom, prenom, age, numeroTelephone, adresse, ville, typeVehicule, notePonctualite, noteConvivialite, noteVehicule, noteGlobale) CLIENTS(id, nom, prenom, age, numeroTelephone, adresse, ville, moyenPaiement) COURSES(idChauffeur, idClient, dateHeureDepart, dateHeureArrivee, adresseDepart, adresseArrivee, montantFacture, notePonctualite, noteConvivialite, noteVehicule, noteGlobale, observations) Le nom des attributs est explicite. Un chauffeur conduit son propre véhicule. Les notes associées à une course sont données par le client de la course. Toutes ou certaines de ces notes peuvent être absentes. Les notes associées à un chauffeur sont des moyennes des notes correspondantes obtenues pour toutes les courses notées de ce chauffeur.

(a) Pour la table COURSES, identifiez la clé primaire et les clés étrangères (0,5 points)

Correction :
Clé primaire: (idChauffeur, idClient, dateHeureDepart)
Clés étrangères: idChauffeur→CHAUFFEURS.id, idClient→CLIENTS.id

(b) Donnez la commande SQL pour créer la table COURSES. (0.5 point)

Correction :

```
CREATE TABLE COURSES(
    idChauffeur NUMBER(10) not null,
    idClient NUMBER(10) not null,
    dateHeureDepart DATE not null,
    dateHeureArrivee DATE not null,
    adresseDepart VARCHAR(100),
    adresseArrivee VARCHAR(100),
    montantFacture NUMBER(6,2),
    notePonctualite INTEGER,
    noteConvivialite INTEGER,
    noteVehicule INTEGER,
    noteGlobale INTEGER,
    observations VARCHAR(256),
    CONSTRAINT pk_course
        PRIMARY_KEY (idChauffeur, idClient, dateHeureDepart),
```

```
CONSTRAINT fk_chauffeur
  FOREIGN KEY (idChauffeur) REFERENCES CHAUFFEURS(id),
CONSTRAINT fk_course
  FOREIGN KEY (idClient) REFERENCES CLIENTS(id))
```

(c) (Algèbre et SQL) Quels sont les noms des clients qui ont donné au chauffeur `idChauffeur=13` une note de ponctualité inférieure à 10 ? (1 point)

Correction :
Algèbre: $\pi_{nom}(\sigma_{idChauffeur=13 \wedge noteConvivialite \leq 10}(COURSES \bowtie_{idClient=id} CLIENT))$
SQL: SELECT idClient FROM COURSES, CLIENT WHERE idClient = id AND idChauffeur = 13 AND noteConvivialite <= 10;

(d) (SQL seulement) Quel est le nom du chauffeur (ou des chauffeurs) de Paris qui a fait le plus de courses ? (2 points)

Correction :

```
SELECT nom
FROM COURSES, CHAUFFEURS
Where id = idChauffeur and ville = 'Paris'
GROUP BY idChauffeur
HAVING COUNT(*) = (SELECT MAX(COUNT(*))
                   FROM COURSES
                   GROUP BY idChauffeur)
```

(e) (Algèbre seulement) Donnez le id du client (ou des clients) qui a effectué la course la plus chère ? (1 point)

Correction :
COURSES' = $\rho_{attribut \rightarrow attribut'}(COURSES)$
Le montant facture le plus grand:
MMAX =$\pi_{montantFacture}(COURSES) - \pi_{montantFacture}(COURSES \bowtie_{montantFacture < montantFacture'} COURSES')$
Résultat:
R = $\pi_{idClient}(COURSE \bowtie MMAX)$

(f) (SQL) Quels sont les id des clients de Tours qui ont donné une note globale moyenne inférieure à 10 aux chauffeurs qu'ils ont notés ? Triez ces clients en ordre croissant de la moyenne de la note globale. (2 points)

Correction :
SELECT cl.nom FROM CLIENT cl, COURSES co WHERE cl.id = co.id$_c lientANDcl.ville =' Tours'GROUPBYid_c lientHAVINGAVG(noteGlobale) < 10ORDERBYAVG(noteGlobale)$;

(g) (Algèbre) Donnez les id des clients qui ont effectué au moins une course avec chaque chauffeur ? (1 points)

Correction :
$A = \pi_{idClient,idChauffeur}(CHAUFFEURS)$
$B = \rho_{id \rightarrow idChauffeur}(\pi_{id}(CHAUFFEURS))$
$Resultat = A \div B$

(h) (Algèbre + SQL) Donnez les id de chauffeurs qui n'ont transporté que des clients plus âgés que eux-mêmes. (2 points)

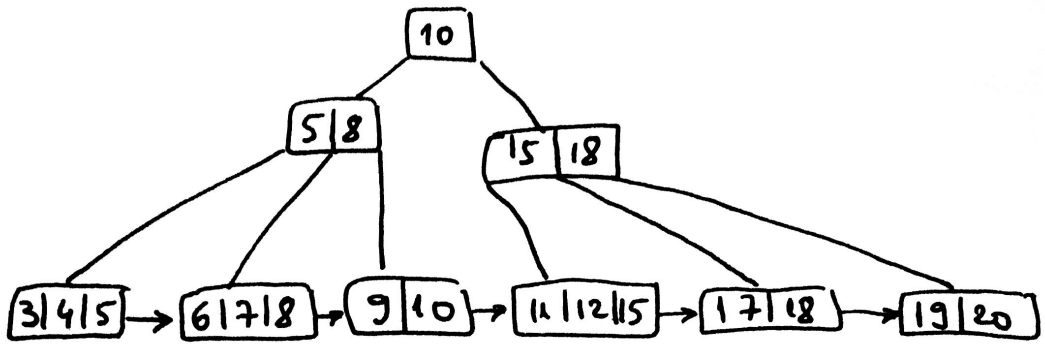
Correction :
$CH = \rho_{id \rightarrow idChauffeur, age \rightarrow ageChauffeur}(CHAUFFEURS)$
$CL = \rho_{id \rightarrow idClient, age \rightarrow ageClient}(CLIENTS)$
$IDCH = \rho_{id \rightarrow idChauffeur}(\pi_{id}(CHAUFFEURS))$
$Res = IDCH - \pi_{idChauffeur}(\sigma_{ageChauffeur >= ageClient}(CL \bowtie (COURSE \bowtie CH)))$
SELECT id FROM CHAUFFEURS WHERE id NOT IN (SELECT ch.id FROM CHAUFFEURS ch, COURSES co, CLIENTS cl WHERE ch.id = co.idChauffeur AND cl.id = co.idClient AND ch.age >= cl.age)

13.2 Index (3 points)

(a) Créer un index en arbre B+ d'ordre 2 sur la note globale de chauffeurs. Les valeurs des clés sont données ci-dessous. Montrez chaque étape d'éclatement de l'arbre. (2 points)

noteGlobale : 10, 5, 3, 17, 18, 4, 7, 6, 9, 8, 19, 11, 20, 15, 12

Correction :



(b) Combien de nœuds faut-il traverser pour faire une lecture des valeurs entre 6 et 15 (dans l'arbre crée au point précédent)? (1 point)
Correction :
Il faut 3 lectures de nœud pour descendre dans les feuilles et ensuite il faut lire encore 2 nœuds dans la liste chaînée pour un total de 5 nœuds.

13.3 Optimisation (4 points)

Soit la requête suivante:

```
SELECT cl.nom
FROM COURSE co, CHAUFFEURS ch, CLIENT cl
WHERE co.idChauffeur = ch.id AND co.idClient = cl.id
      AND dateHeureDepart > 23H00 and ch.noteGlobale < 10;
```

On suppose qu'il y a *uniquement* des index sur CHAUFFEURS(noteGlobale) et COURSES(idChauffeur).

(a) Expliquez en quelques mots ce que fait la requête. (0.5 points)
Correction :
La requête affiche les noms des clients qui font des courses après 23H00 avec un chauffeur qui a une note globale inférieure à 10.

(b) Donnez un plan d'exécution (arborescente ou EXPLAIN) produit par l'optimiseur pour la requête. **Argumentez** votre réponse. (2.5 points)
Correction :
L'index CHAUFFEURS(noteGlobale) permet de chercher directement les chauffeurs qui ont noteGlobale < 10 et ensuite récupérer leur id, qu'on cherche ensuite dans la table COURSES en utilisant l'index COURSES(idChauffeur). Il faut ensuite parcourir la table CLIENT en séquentiel pour récupérer les noms des clients. (1 point)

```
0 SELECT STATEMENT
1   MERGE JOIN
2     SORT JOIN
5      NESTED LOOPS
4        TABLE ACCESS BY ROWID CHAUFFEURS
5          INDEX RANGE SCAN CHAUFFEURS_NOTEGLOBALE_IDX
6        TABLE ACCES BY ROWID COURSE
7          INDEX RANGE SCAN COURSES_IDCHAUFFEUR_IDX
8     SORT JOIN
9     TABLE ACCESS FULL CLIENTS
```

(c) Peut-on accélérer encore la requête en ajoutant des index supplémentaires? (1 point)
Correction :

La présence du MERGE JOIN indique qu'il manque un index dans la table CLIENTS. Un index sur CLIENTS(id) permettrais de chercher directement idClients sans recourir à un parcours séquentiel.

13.4 Concurrence (3 points)

Soit l'exécution concurrente suivante :

$H = r_1(x)r_2(y)w_3(z)r_1(y)w_1(x)r_3(x)w_2(y)C_1w_3(x)w_2(z)C_3w_2(y)C_2$

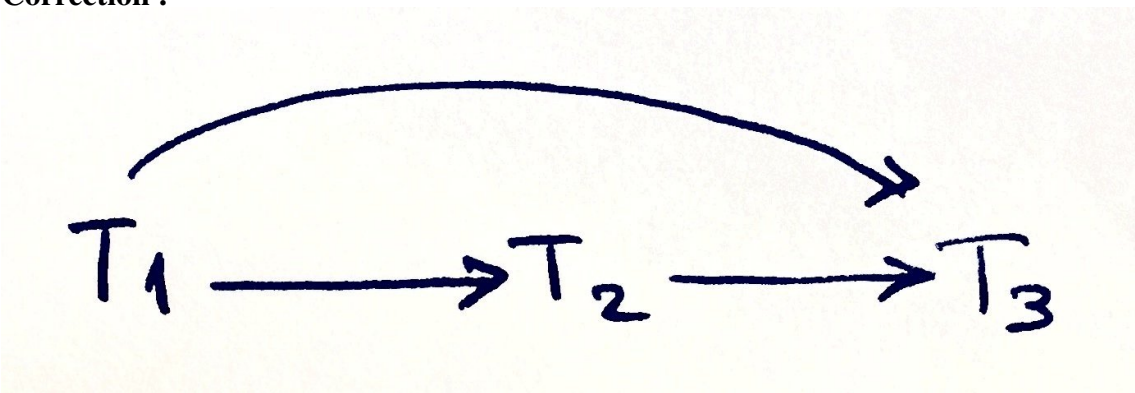
(a) Donnez la liste des conflits de H. (0.5 points)
Correction :
Les conflits:

en x : $r_1w_3 \quad w_1r_3 \quad w_1w_3$

en y : r_1w_2

en z : w_2w_3

(b) Donnez le graphe de sérialisation de H. Que pouvez-vous déduire à partir de ce graphe? (0.5 points)
Correction :



Comme il n'y a pas de cycles on en déduit que l'exécution est sérialisable.

(c) Donnez l'histoire finale après l'application de l'algorithme de verrouillage à deux phases. Donnez les détails du déroulement de l'algorithme. (2 points)
Correction :
$H' = r_1(x)r_2(y)w_3(z)r_1(y)w_1(x)C_1r_3(x)w_2(y)w_3(x)C_3w_2(z)w_2(y)C_2$

14 | NFP 107 HTT, Septembre 2015

14.1 Algèbre relationnelle et SQL (10 points)

Voici une partie de la base de données d'un site gérant la réservation de billets pour des concerts :

```
FORMATION(nom, annéeFormation, nombreMembres, pays)
SALLE(nomSalle, typeSalle, ville, capacité)
CONCERT(codeConcert, nomFormation, nomSalle, dateConcert, nbBilletsVend
BILLET(idBillet, codeConcert, prix)
```

Pour une formation sont stockés son nom, l'année où l'artiste/le groupe a débuté, le nombre de membres composant le groupe (1 pour un artiste en solo) et son pays d'origine. Pour la salle nous connaissons son nom, son type (stade, arène, petite salle, grande salle, etc), sa ville et son nombre de places. Les concerts sont identifiés par un code, donnés par une formation, se déroulent dans une salle, ont lieu à une date et ont un nombre de billets vendus. Les billets ont un identifiant, un code concert et un prix.

(a) Pour la table CONCERT, identifiez la clé primaire et les clés étrangères (0,5 points)

Correction :
Clé primaire: codeConcert
Clés étrangères: nomSalle→SALLE.nomSalle, nomFormation→FORMATION.nom

(b) Donnez la commande SQL pour créer la table CONCERT. (0.5 point)

Correction :

```
CREATE TABLE CONCERT(
    codeConcert NUMBER(10),
    nomFormation VARCHAR(50),
    nomSalle VARCHAR(50)
    dateConcert DATE,
    nbBilletsVendus NUMBER(10),
    CONSTRAINT pk_concert PRIMARY_KEY (codeConcert),
    CONSTRAINT fk_concert FOREIGN_KEY (nomSalle) REFERENCES SALLE(no
    CONSTRAINT fk_concert FOREIGN_KEY (nomFormation) REFERENCES FORM
```

(c) (Algèbre et SQL) Donnez toutes les salles de Paris qui ont plus de 2000 places. (1 point)

Correction :
Algèbre: $\pi_{nomSalle}(\sigma_{ville=Paris \wedge capacite \geq 2000}(SALLE)$

```
SELECT nomSalle
FROM SALLE
WHERE ville = 'Paris' AND capacité > 2000;
```

(d) (Algèbre et SQL) Donnez les pays des formations dont au moins un billet coûte plus de 200 euros. (2 points)

Correction :
Algèbre: $\pi_{pays}((BILLET \bowtie CONCERT) \bowtie_{nomFormation=nom} FORMATION)$

```
SELECT pays
FROM FORMATION f, CONCERT c, BILLET b
WHERE b.codeConcert = c.codeConcert AND C.nomFormation = f.nom AND p
```

(e) (SQL seulement) Donnez le nom des formations qui ont fait la recette (rentrée d'agent) la plus grande tous concerts confondus en septembre 2015 au Stade de France. (2 points)

Correction :

```
SELECT nomFormation
FROM SALLE s, CONCERT c, BILLET b
WHERE s.nomSalle = c.nomSalle AND c.codeConcert = b.codeConcert
      AND dateConcert BETWEEN '01/09/2015' AND '30/09/2015';
GROUP BY nomFORMATION
HAVING SUM(nbBilletsVendus*prix) =
             (SELECT MAX(SUM(nbBilletsVendus*prix))
              FROM SALLE s, CONCERT c, BILLET b
              WHERE s.nomSalle = c.nomSalle
                    AND c.codeConcert = b.codeConcert;
                    AND dateConcert BETWEEN '01/09/2015' AND '30/09/
              GROUP BY nomFORMATION)
```

(f) (Algèbre et SQL) Donnez le nom des formations qui n'ont jamais donné un concert dans une salle plus petite de 10000 places. (2 points)

Correction :
Algèbre: $\pi_{nom}(FORMATION) - \rho_{nomFormation \rightarrow nom}(\pi_{nomFormation}(\sigma_{capacit<10000}((CONCERT \bowtie SALLE)))$

```
SELECT nom
FROM FORMATION
```

```
WHERE nom NOT IN
       SELECT nomFORMATION
       FROM CONCERT c, SALLE s
       WHERE c.nomSalle = s.nomSalle AND capacité < 10000;
```

(g) (Algèbre seulement) Donnez le nom des formations qui ont joué dans toutes les salles. (Attention : deux salles situées dans des villes différentes peuvent avoir le même nom) (1 point)

Correction :
$A = \pi_{nomFormation,nomSalle,Ville}(CONCERT \bowtie SALLE)$
$B = \pi_{nomSalle,Ville}(SALLE)$
$Resultat = A \div B$

(h) (Algèbre seulement) Donnez le nom de la formation qui a vendu le plus de billets pour un concert. (1 point)

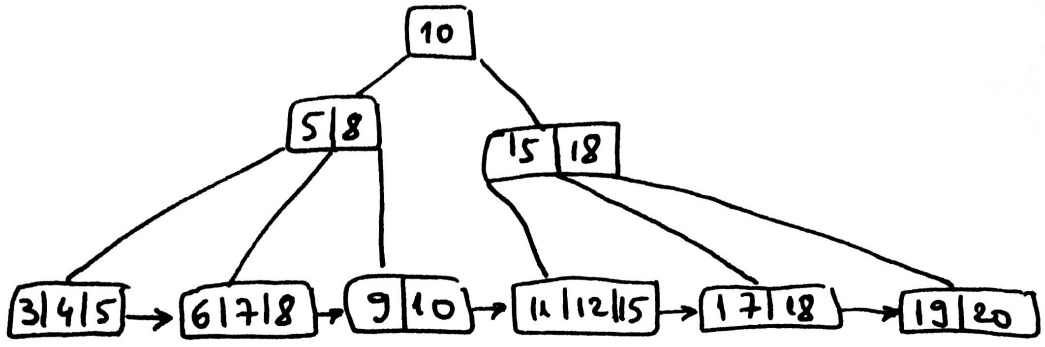
Correction :
$CONCERT' = \rho_{attribut \rightarrow attribut'}(CONCERT)$
$\pi_{nomFormation}(CONCERT) - \pi_{nomFormation}(CONCERT \bowtie_{nbBilletsVendus < nbBilletsVendus'} CONCERT')$

14.2 Index (3 points)

(a) Créer un index en arbre B d'ordre 2 sur le prix des billets. Les valeurs des clés sont données en bas. Montrez chaque étape d'éclatement de l'arbre. (2 points)

prix : 100, 95, 110, 120, 80, 85, 130, 90, 75, 125, 115, 60, 70, 140, 135, 65, 145

Correction :



(b) Combien de nœuds faut-il traverser pour faire une lecture par intervalle des valeurs entre 80 et 130 (dans l'arbre créé au point précédent) ? L'utilisation d'un arbre B+ peut améliorer cette valeur ? Justifiez. (1 point)

Correction :
On traverse l'arbre de gauche a droite en suivant les fléchés rouges, donc on visite 12 nœuds (parfois un nœud est visité plusieurs fois). L'arbre B+ peut améliorer beaucoup cette valeur, car une fois descendu dans les feuilles il suffit de lire à droite dans la liste chaînée les valeurs successives.

14.3 Optimisation (4 points)

Soit la requête suivante :

```
SELECT nbBilletsVendus
FROM SALLE s, CONCERT c
WHERE s.nomSalle = c.nomSalle AND s.ville='Paris';
```

On suppose qu'il y un index sur l'attribut nomSalle dans la table CONCERT.

(a) Donnez un plan d'exécution (arborescente ou EXPLAIN) produit par l'optimiseur pour la requête. **Argumentez** votre réponse. (2 points)

Correction :

L'index CONCERT(nomSalle) permet de faire une recherche sur nomSalle. La clé à chercher vient donc de l'autre table, SALLE, qui doit être parcourue en séquentiel.

```
0 SELECT STATEMENT
1   NESTED LOOPS
2     TABLE ACCESS FULL SALLE
3     TABLE ACCESS BY ROWID CONCERT
5          INDEX RANGE SCAN CONCERT_NOMSALLE_IDX
```

(b) Comment peut-on accélérer encore la requête en ajoutant des index supplémentaires ? Donnez le plan d'exécution (arborescente ou EXPLAIN). (2 points)

Correction :
On peut éliminer le parcours séquentiel de la table SALLE, car la requête s'intéresse seulement aux salles situées à Paris. Pour cela on ajoute un index sur SALLE(ville) et le plan EXPLAIN devient :

```
0 SELECT STATEMENT
1   NESTED LOOPS
3     TABLE ACCESS BY ROWID SALLE
4          INDEX RANGE SCAN SALLE_VILLE_IDX
5     TABLE ACCESS BY ROWID CONCERT
6          INDEX RANGE SCAN CONCERT_NOMSALLE_IDX
```

14.4 Concurrence (3 points)

Soit l'histoire d'exécution suivante :

$H = r_3(z)w_2(x)w_3(z)r_1(x)r_2(y)w_1(x)r_3(y)r_1(x)c_3r_2(z)w_2(x)c_2c_1$

(a) Donnez la liste des conflits de H (0.5 points)

Correction :
Les conflits :
en x : $w_2r_1 \quad w_2w_1 \quad r_1w_2 \quad w_1w_2$
en y : pas de conflit
en z : w_3r_2

(b) Donnez le graphe de sérialisation de H. Que pouvez vous déduire à partir de ce graphe ? (0.5 points)

Correction :

Comme il y a des cycles on déduit que que l'exécution n'est pas sérialisable.

(c) Donnez l'histoire finale après l'application de l'algorithme de verrouillage à deux phases. Donnez les détails du déroulement de l'algorithme. On suppose que les verrous sont relâchés immédiatement après les *commit. (2 points)*

Correction :
$H' = r_3(z)w_2(x)w_3(z)r_2(y)r_3(y)c_3r_2(z)w_2(x)c_2r_1(x)w_1(x)r_1(x)c_1$